

RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning

Zihan Wang^{*1}, Kangrui Wang^{*1}, Qineng Wang^{*1}, Pingyue Zhang^{*1}, Linjie Li^{*2}, Zhengyuan Yang⁴, Xing Jin⁶, Kefan Yu¹, Minh Nhat Nguyen⁷, Licheng Liu¹, Eli Gottlieb¹, Yiping Lu¹, Kyunghyun Cho⁵, Jiajun Wu³, Li Fei-Fei³, Lijuan Wang⁴, Yejin Choi³, Manling Li¹

¹Northwestern University ²University of Washington ³Stanford University ⁴Microsoft
⁵New York University ⁶University of British Columbia ⁷Singapore Management University

Training large language models (LLMs) as interactive agents presents unique challenges including **long-horizon decision making** and **interacting with stochastic environment feedback**. While reinforcement learning (RL) has enabled progress in static tasks, multi-turn agent RL training remains underexplored. We propose **StarPO** (State-Thinking-Actions-Reward Policy Optimization), a general framework for trajectory-level agent RL, and introduce **RAGEN**, a modular system for training and evaluating LLM agents. Our study on four stylized environments reveals three core findings. **First**, our agent RL training shows a recurring mode of **Echo Trap** where **reward variability cliffs and gradient spikes**; we address this with **StarPO-S**, a stabilized variant with **trajectory filtering, critic incorporation, and gradient stabilization**. Second, we find the **shaping of RL rollouts** would benefit from **diverse initial states, medium interaction granularity and more frequent sampling**. Third, we show that **without fine-grained, reasoning-aware reward signals, agent reasoning hardly emerge** through multi-turn RL and they may show shallow strategies or hallucinated thoughts.

Keywords: LLM Agents, Multi-turn RL

Website: <https://ragen-ai.github.io/>

Code/Environments: <https://github.com/RAGEN-AI/RAGEN>.

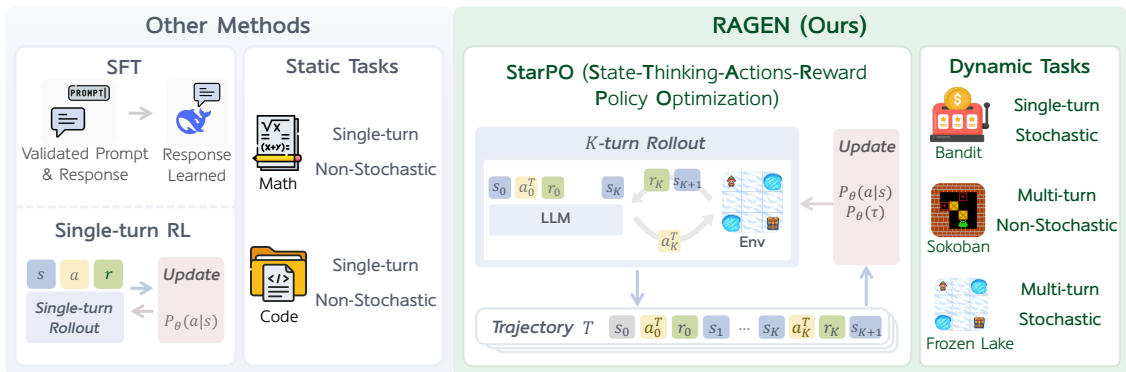


Figure 1 | Previous methods focus on **non-interactive tasks** such as math or code generation. **RAGEN** implements StarPO, a general agent RL framework that supports multi-turn rollouts, trajectory-level reward assignment, and policy updates, on agent tasks **requiring multi-turn stochastic interaction**.

1. Introduction

Training large language models (LLMs) to function as autonomous agents in interactive environments presents unique challenges. Unlike static tasks such as single-turn math problem solving (Shao et al., 2024a) or coding (DeepSeek-AI et al., 2024), agent settings require models to **make sequential decisions, maintain memory across turns, and adapt to stochastic feedback from their environment**. These settings—central to planning assistants, robotics, and tutoring agents—demand that **models not only perform well, but also self-improve through experience**.

While recent work has explored reinforcement learning (RL) for LLMs (DeepSeek-AI et al., 2025; Gao et al., 2024; Kumar et al., 2024; OpenAI, 2024; Pan et al., 2025; Zeng et al., 2025c) using rule-based reward, it remains largely **underexplored to train interactive agents that self-evolve to reason and adapt through rule-based RL**. In particular, LLM agent training often exhibits training instability, complex reward signals, and limited generalization across environment changes, especially under multi-turn interaction with stochastic feedback. A key open question is: *what design factors make self-evolving LLM agents learn effectively and stably?*

We explore this question through a systematic study of agent learning under a general RL framework **StarPO (State-Thinking-Actions-Reward Policy Optimization)**. StarPO provides a unified view of **multi-turn, trajectory-level agent training with flexible control over reasoning, reward assignment, and prompt-rollout structure**. Built on top of StarPO, we develop **RAGEN, a modular agent training and evaluation system** designed to support the study of RL-based training in LLM Agents. RAGEN implements the full training loop—including rollout generation, reward assignment, and trajectory optimization—serving as a research infrastructure for systematic analysis of LLM agent training dynamics under multi-turn and stochastic environments.

Training LLM agents on real-world tasks like web browsing often depends on **pretrained priors and heavy task-specific engineering**. We evaluate RAGEN on four environments spanning different complexities: **Bandit** (single-turn, stochastic), **Sokoban** (multi-turn, deterministic), **Frozen Lake** (multi-turn, stochastic), and **WebShop** (multi-turn, open-domain). The first three symbolic environments are **minimalistic and fully controllable**, while WebShop adds **real-world understanding and reasoning**. Together, they enable analysis of generalization across varied decision-making challenges.

Using this setup, we analyze **three key dimensions of agent learning**, and summarize below findings that **reveal core challenges and design principles** for stable agent RL training:

1. **Gradient Stability in Multi-turn RL is the Key to Stable Training.** We find that **multi-turn RL training** often leads to a **recurring instability pattern, Echo Trap**, where agents overfit to locally rewarded reasoning patterns, marked by reward variability collapse, entropy drop, and gradient spikes. To mitigate this failure mode, we propose **StarPO-S**, a **stabilized variant of our framework** that improves learning robustness through variability-based trajectory filtering, critic baselining, and decoupled clipping.
2. **Rollout Frequency and Diversity Shape Self-Evolution.** In RL-based agent training, LLM self-generated rollout trajectories are served as core training material. We identify key rollout factors for stable agent RL training: (1) ensuring that **rollouts come from diverse initial states** with **multiple responses per initial state**, (2) **implementing multiple actions each turn** to improve interaction horizon within fixed turn limit, (3) **maintaining a high rollout frequency** to ensure online feedback reflects current policies.
3. **Emerging Agent Reasoning Requires Meticulous Reward Signal.** We find that simply encouraging reasoning in the action format does not guarantee reasoning behavior. Even

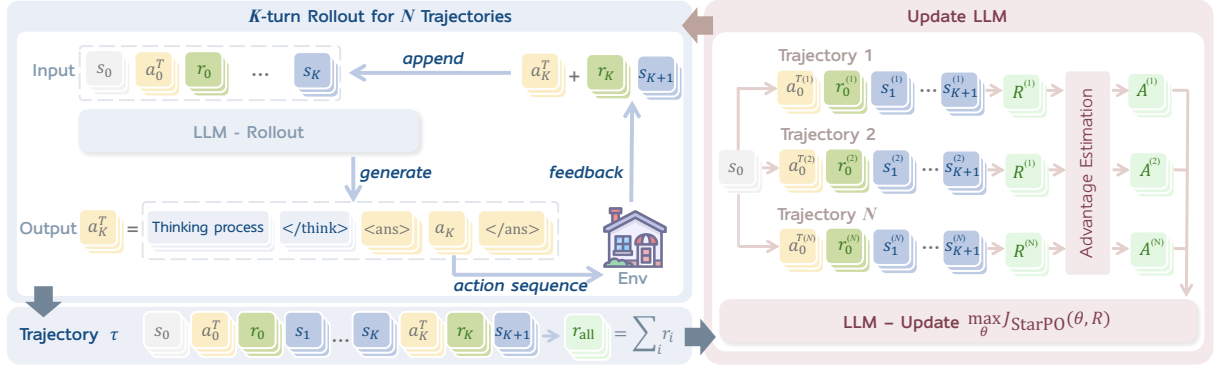


Figure 2 | The State-Thinking-Actions-Reward Policy Optimization (StarPO) framework. LLM generates reasoning-guided actions for multi-turn interactions with environments and accumulates trajectory-level rewards, normalized and used to update the LLM policy.

when models are prompted to reason (e.g., with ‘<think>’ tokens) with trajectory-level optimization via StarPO, they often **regress to direct action selection if reasoning offers no distinct reward advantage**. We assume this is due to the simple action spaces in MDP where shallow strategies suffice. Moreover, when rewards only reflect task success, models produce **hallucinated reasoning**, revealing a mismatch between thoughts and environment states. These issues underscore the need for **fine-grained, reasoning-aware reward design for long-horizon agent training**.

2. Framework

2.1. The MDP Formulation for Agent Training

Previous reinforcement learning (RL) for language models often assumes a single-turn setting, where the goal is to **maximize the expected reward $R(s, a)$ over prompt-response pairs (s, a) sampled from a dataset $\mathcal{D}$** :

$$J_{\text{step}}(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot|s)} [R(s, a)]. \quad (1)$$

However, LLM-based agents must operate in interactive environments that **unfold over multiple turns and exhibit stochastic feedback**. To capture these dynamics, we formulate the problem as a **Markov Decision Process (MDP)** $\mathcal{M} = \{S, A, P\}$, where S represents states (e.g., observation sequences or interaction histories), A represents actions (often token sequences), and P denotes the transition dynamics and reward generation process. The agent policy π_{θ} generates an action a_t at each time step t , conditioned on the current state s_t and the interaction history. The environment returns a reward r_t and a new state s_{t+1} given the current transition dynamics:

$$a_t \sim \pi_{\theta}(\cdot|s_t, \tau_{<t}), \quad (r_t, s_{t+1}) \sim P(\cdot|s_t, a_t),$$

where $\tau_{<t} = \{s_0, a_0, r_0, \dots, s_{t-1}, a_{t-1}, r_{t-1}\}$ denotes the **interaction history**. This interactive process continues for a maximum horizon K , yielding a full trajectory $\tau = \{s_0, a_0, r_0, \dots, s_K\}$ that forms the learning material for the agent.

2.2. StarPO: Reinforcing Reasoning via Trajectory-Level Optimization

We introduce **StarPO (State-Thinking-Action-Reward Policy Optimization)**, a general RL framework designed to optimize entire multi-turn interaction trajectories for LLM agents. Unlike

previous methods for static tasks that treat each action independently, **StarPO treats the entire trajectory**—including **observations, reasoning traces, actions, and feedback**—as a coherent unit for rollout and model optimization. The objective is to **maximize expected trajectory reward**:

$$J_{\text{StarPO}}(\theta) = \mathbb{E}_{\mathcal{M}, \tau \sim \pi_\theta} [R(\tau)], \quad (2)$$

where $\mathcal{M}$ is the MDP, τ is a full sequence of reasoning-augmented interactions, and $R(\tau)$ denotes the cumulative reward over the entire trajectory. The policy probability $\pi_\theta(\tau)$ is decomposed into token-level likelihoods, making StarPO directly compatible with autoregressive LLMs. Figure 2 illustrates the full StarPO process, and we break them down in detail below.

Trajectory-Level Objective in StarPO vs. Previous Methods

Previous methods (e.g., PPO (Schulman et al., 2017), GRPO (Shao et al., 2024a)):

$$J_{\text{step}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [R(x, y)] \quad (\text{optimize single-turn output } y \text{ given input } x)$$

StarPO (ours):

$$J_{\text{StarPO}}(\theta) = \mathbb{E}_{\mathcal{M}, \tau \sim \pi_\theta} [R(\tau)] \quad (\text{optimize total reward over trajectory } \tau = \{s_0, a_0, r_0, \dots, s_K\})$$

2.2.2. Optimization Procedure: Learning from Reasoning-Interaction Trajectories

At each training iteration, the agent begins from an initial state s_0 and **generates N trajectories**. At each step t , the agent produces **a reasoning-guided structured output**:

$$a_t^T = \langle \text{think} \rangle \dots \langle \text{think} \rangle \langle \text{answer} \rangle a_t \langle \text{answer} \rangle, \quad (3)$$

where a_t^T is the full action output including intermediate reasoning, and a_t is a sequence of environment-executable sub-action. The environment then returns the next state s_{t+1} and reward r_t . The rollout stage produces complete trajectories $\tau = \{s_0, a_0^T, r_0, s_1, \dots, a_{K-1}^T, r_{K-1}, s_K\}$, where *every component is LLM-generated or environment-induced* and will be jointly optimized.

StarPO interleaves rollout and update steps. **New rollouts can be generated on-policy using π_θ , or sampled from a replay buffer under π_{old}** . Each training loop consists of P initial states s_0 , each generating N trajectories, and updates are performed with batch size E for L total loops. This yields $S = \frac{L \cdot P \cdot N}{E}$ total gradient update steps. Additional training mechanisms are discussed in §3.

2.2.3. Modular Optimization Strategies

StarPO supports a variety of policy optimization algorithms under **a unified trajectory-level abstraction**. For each rollout trajectory $\tau_i = \{\tau_{i,(1)}, \dots, \tau_{i,(|\tau_i|)}\}$ of totally $|\tau_i|$ tokens, we instantiate StarPO with the following optimization strategies for token-level updates:

- **PPO (Schulman et al., 2017)**. We use the PPO objective (More details can be found in Appendix A), where a critic is trained to estimate token-level value and advantages $A_{i,t}$:

$$J_{\text{PPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min \left[\frac{\pi_\theta(\tau_{i,(t)}|\tau_{i,<t})}{\pi_{\text{old}}(\tau_{i,(t)}|\tau_{i,<t})} \cdot A_{i,t}, \text{clip} \left(\frac{\pi_\theta(\tau_{i,(t)}|\tau_{i,<t})}{\pi_{\text{old}}(\tau_{i,(t)}|\tau_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \cdot A_{i,t} \right], \quad (4)$$

where G is the number of trajectories in the batch, $\tau_{i,(t)}$ denotes the t -th token in trajectory τ_i , and $\tau_{i,<t}$ is its prefix.

- **GRPO (Shao et al., 2024a)**. For critic-free training leveraging GRPO, we assign a scalar reward $R(\tau_i)$ to each trajectory and normalized advantage $\hat{A}_{i,t}$ across all tokens in τ_i :

$$\hat{A}_{i,t} = \frac{R(\tau_i) - \text{mean}(\{R(\tau_1), \dots, R(\tau_G)\})}{\text{std}(\{R(\tau_1), \dots, R(\tau_G)\})}. \quad (5)$$

The GRPO objective becomes:

$$J_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min \left[\frac{\pi_{\theta}(\tau_{i,t} | \tau_{i,<t})}{\pi_{\text{old}}(\tau_{i,t} | \tau_{i,<t})} \cdot \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(\tau_{i,t} | \tau_{i,<t})}{\pi_{\text{old}}(\tau_{i,t} | \tau_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_{i,t} \right]. \quad (6)$$

2.3. The RAGEN System

To implement StarPO in practice, we build **RAGEN**, a **complete system for LLM agent training** in controlled environments. RAGEN supports **structured rollouts, customizable reward functions, and integration with multi-turn, stochastic environments**. It serves both as the execution backend for StarPO and as a platform for studying stability, generalization, and learning dynamics in training reasoning agents. RAGEN is designed to be **extensible: new environments, reward schemes, or rollout strategies can be easily plugged in**, serving as a foundation for RL-based agent training.

3. Experiment Setup

3.1. Environments and Tasks

We evaluate LLM agents on four environments spanning symbolic and realistic decision-making: **Bandit** tests **risk-sensitive reasoning under noisy feedback**; **Sokoban** requires **irreversible symbolic planning**; **Frozen Lake** combines planning with probabilistic transitions; and **WebShop** involves **natural language grounding and web environment interaction**. The first three symbolic environments are deliberately minimal and fully controllable to support clean analysis, while WebShop introduces realistic task structure and language input. Environment visualizations are in Appendix C.1.

3.2. Training Settings

In our main experiments, we train Qwen-2.5 Instruct **0.5B models** for **three symbolic tasks** and its **3B** variant for the **challenging WebShop**. We also report various model performance in Appendix D. Models are trained with StarPO variants on H100 GPUs for 100-200 rollout-update iterations. Each batch samples $P=8$ prompts, with $N=16$ rollouts per prompt, up to 5 turns and 10 actions. Policy updates use GRPO or PPO with GAE ($\gamma=1.0, \lambda=1.0$), Adam optimizer, entropy bonus ($\beta=0.001$), and a response-format penalty (-0.1). More details can be found in Appendix C.2.

3.3. Evaluation Metrics

We evaluate on **256 fixed prompts per environment** with temperature $T=0.5$, **truncating episodes after 5 turns**. Metrics include: (i) success rate (task completion), (ii) rollout entropy (exploration), (iii) in-group reward variability (behavioral diversity), (iv) response length (reasoning verbosity), and (v) gradient norm (training stability). All are computed over **validation instances**. More details can be found in Appendix C.3.

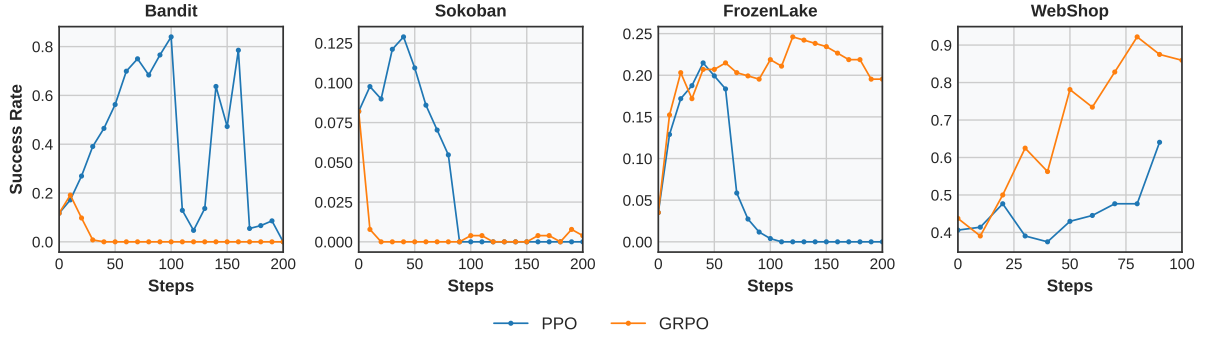


Figure 3 | **Baseline StarPO performance across environments.** Symbolic tasks like Bandit and Sokoban lead to collapse while real-world WebShop starts high and improves fast. PPO is stronger in Bandit and Sokoban to provide more stable token-level reward signal, while GRPO is stronger in Frozen Lake whose randomness makes the **state value hard to estimate**, and WebShop whose **strong initial performance** lessens the need for a critic to stabilize gradients.

4. Experimental Results and Findings

4.1. Multi-turn Agent RL Training Introduces New Instability Pattern

We evaluate baseline StarPO across agent tasks (Figure 3). **Symbolic environments** like Bandit and Sokoban **show early improvements** but **eventually collapse**. PPO is more stable than GRPO in these settings, collapsing later and achieving higher performance, likely due to its **critic providing smoother reward estimates**. Interestingly, **GRPO is more stable on Frozen Lake**, likely due to the **difficulty of estimating state values** in this task, which may **destabilize PPO** (see Appendix I). **On WebShop, both methods succeed**, likely due to **strong language prior** and **high initial rewards** reducing the need for a critic.

Finding 1: Single-turn RL may not be directly adapted to Multi-turn agent RL

Vanilla adaptations from single-turn methods like PPO and GRPO **achieve early gains in agent settings but often collapse**. A **critic** in PPO may **delay instability**, but **would not prevent reasoning degradation**, highlighting the need for **specialized stabilization in agent settings**.

To understand the causes of collapse, we compare early- and late-stage trajectories. In the Bandit task, **early-stage trajectories exhibit diverse reasoning** about symbolic meanings and expected rewards, while **later-stage responses become repetitive and deterministic**. This suggests that **RL training may have over-amplified inherent reasoning shortcuts**, **reinforcing locally rewarded templates while suppressing exploration**. We refer to this failure mode as an “**Echo Trap**”, which is similar to the findings in Shumailov et al. (2024), where the model repeatedly reuses memorized reasoning paths when trained on self-generated trajectories, leading to a collapse in diversity and long-term performance degradation. Examples are in Appendix F.

Finding 2: Model collapse in agent RL is reflected as “Echo Trap” over training

We find that early-stage agent respond with diverse symbolic reasoning, but collapse into deterministic, repetitive templates after training. Models converge to fixed phrasing, indicating that RL may **reinforce superficial patterns** instead of general reasoning and forms an “Echo Trap” that hinders long-term generalization.

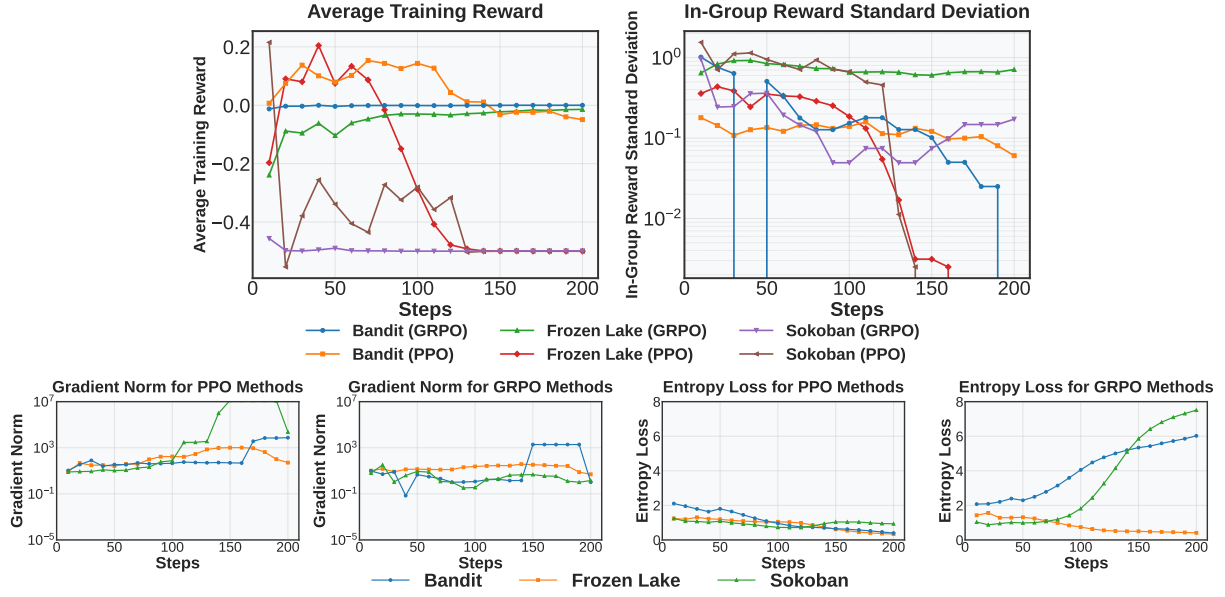


Figure 4 | **Collapse indicators and early warning signals in multi-turn RL.** Average reward and gradient norm (left-side plots) reflect collapse directly, and their plateaus and spikes confirm performance and training instability. **Reward standard deviation and entropy** (right-side plots) often **becomes unstable** before reward degrades, serving as **early warning signals**.

To detect collapse, we monitor two key metrics: (1) **Average Reward**, where **a plateau or drop signals** degraded task performance, and (2) **Gradient Norm**, where **spikes suggest unstable updates**. To identify early signs, we also track rollout-level signals: (1) **Reward Standard Deviation**, indicating whether the policy is producing diverse outcomes or collapsing into repetitive behavior, and (2) **Output Entropy**, which indicates how deterministic the model's predictions are.

Figure 4 summarizes these dynamics across tasks and optimization methods. From the results, we draw the following conclusions regarding **how model collapse emerges in multi-turn agent RL**:

- **Reward standard deviation is an early indicator of convergence.** In FrozenLake-PPO, std drops sharply at step 40, well before reward mean collapses at step 90 while performance is near-optimal. In Bandit-PPO, std bottoms out around step 70, preceding the reward peak at step 120. In Sokoban-PPO, std and mean collapse together near step 10, suggesting early saturation.
- **Gradient norm spikes indicate irreversible collapse.** Once gradient norm spikes emerge, e.g., at step 170 (Bandit), 110 (Sokoban), and 90 (FrozenLake), even small parameter updates could induce drastic loss shifts, after which recovery becomes unlikely.
- **Entropy should follow a stable decay trend during effective learning.** This can be seen from FrozenLake-GRPO. Rapid entropy increases or erratic changes often correlate with collapsed reasoning behavior, such as GRPO on Bandit and Sokoban.

Finding 3: Collapse follows similar dynamics and can be anticipated by indicators

Reward standard deviation and entropy often fluctuate before performance degrades, while **gradient norm spikes typically mark the point of irreversible collapse**. These metrics provide early indicators and motivate the need for stabilization strategies.

These patterns confirm that **multi-turn RL introduces unique challenges that single-turn RL methods fail to handle**. In response, we introduce **StarPO-S**, a stabilized variant that targets sampling quality, gradient stability, and exploration regularization to avoid premature collapse.

4.2. StarPO-S: Stabilize Multi-turn RL with Instance Filtering and Gradient Shaping

To address the instability of multi-turn reinforcement learning, we introduce **StarPO-S**, a **stabilized variant of StarPO** that incorporates three key modifications aimed at **improving training robustness and efficiency**. Building on the insight that declining reward standard deviation often precedes collapse, we investigate the following question: *should agents be trained more intensively on task instances where their behavior is more uncertain with higher reward variability?*

We hypothesize that **the most effective training samples** are those where the agent **exhibits outcome uncertainty**—avoiding both trivial task instances and overly difficult ones. This intuition is rooted in principles of Active Learning (Settles, 2009), where uncertain examples are the most informative ones models should learn from. We define trajectory-level outcome uncertainty U for policy π_θ on a given agent task instance (initial state s_0 in an MDP $\mathcal{M} = \{S, A, P\}$) as:

$$U(\pi_\theta, \mathcal{M}, s_0) = \text{Std}_{\tau \sim \pi_\theta(\cdot|s_0)} [R(\tau)]. \quad (7)$$

During training, we sort prompts based on the standard deviation of reward obtained from repeated rollouts and **retain only the top $p\%$ highly-uncertain prompts at each training step**. Figure 5 shows the effect of varying p in PPO and GRPO under StarPO-S. We further validate the effect of uncertainty-based filtering in Appendix E.

In PPO runs (Figure 5, left), **filtering low-variability rollouts significantly delays collapse**: retaining 75% of rollouts extends stability in FrozenLake from 100 to 140 steps, while 50% avoids collapse entirely. GRPO remains less stable due to its critic-free design but still benefits modestly. Filtering also improves efficiency (Figure 5, right). We adopt 25% as the default for StarPO-S. However, we note that this aggressive value may not be optimal for all scenarios. Tasks like Sokoban and FrozenLake respond well to aggressive filtering, potentially due to their relatively repetitive reasoning patterns and under-representation in pretraining, which make them tend to collapse when similar trajectories dominate the batch. We further present larger model (72B), together with frontier model such as GPT-4o and Qwen-2.5-72B, in Appendix D, to better contextualize our model performance.

Finding 4: Filtering low-variance trajectories improves stability and efficiency

Training on high-variance prompts delays or eliminates collapse in multi-turn RL. StarPO-S **improves performance and reduces update steps** by discarding low-information rollouts, especially under PPO. This aligns with active learning principles, where uncertain examples offer the most informative learning signals.

In addition to uncertainty-based filtering, we adopt **two gradient shaping techniques** inspired by DAPO (Yu et al., 2025) designed for single-turn RL: **KL Term Removal** and **Clip-Higher** (Asymmetric Clipping). We extend and evaluate them in the multi-turn agent setting, and find both methods **boost the success rate and extend stable training phases**, showing how multi-turn RL benefits from more flexible gradient shaping. Design details and performance ablation is in Appendix D.

Overall Comparison. We compare StarPO-S with vanilla StarPO across three tasks in Figure 6. **StarPO-S consistently delays collapse and enhances final task performance**. We attribute these

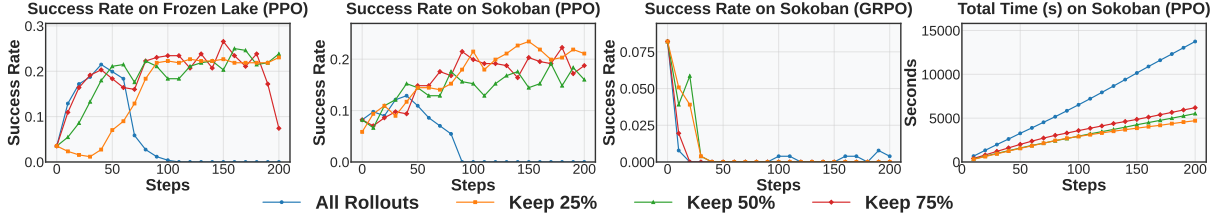


Figure 5 | **Effect of uncertainty-based filtering on multi-turn RL stability.** Filtering out low-variability trajectories reduces collapse risk and improves success rate. On PPO variants, **collapse is largely mitigated when more than half of the trajectories are filtered.** **Training time is reduced.**

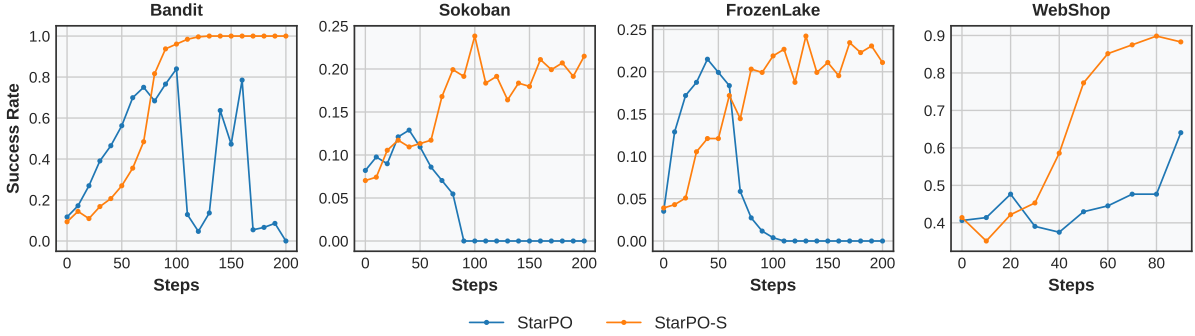


Figure 6 | **StarPO-S improves stability and final performance across tasks.** Compared to vanilla StarPO, StarPO-S relieves collapse in all four tasks and could achieves higher success rates.

gains to **more selective training data** (via uncertainty filtering), **more balanced optimization signals** (via KL removal and decoupled clipping), **reducing narrowed reasoning modes**. In the appendix D, we further discuss about other variants that may stablize training and enhance performance, such as selective response mask and Bi-level Geganal Advantage Estimation (GAE) Wang* et al. (2025).

4.3. Generating Useful Trajectories for RL Training

Effective RL training depends heavily on trajectory quality. We study three key rollout dimensions—*task diversity*, *interaction granularity*, and *rollout frequency*—by training on vanilla Sokoban and evaluating on other tasks including SokobanNewVocab, LargeSokoban, and FrozenLake Task, which we detail in Appendix K.

Higher task diversity with response comparison improves generalization. Task diversity refers to the number of distinct prompts used for each rollout-update cycle. With a fixed batch size, it trades off against the number of responses per prompt. In our experiments (Table 1), we vary this trade-off and find that higher task diversity—achieved by fewer responses per prompt (e.g., 4 per prompt)—consistently yields better generalization. This only holds when each prompt includes multiple rollouts, enabling the agent to contrast different outcomes under similar conditions.

Allowing more action budgets enables planning, while overly long-horizon rollouts inject noise. We vary the number of actions allowed per turn in Table 2. Allowing up to 5 or 6 actions per turn yields the best performance, especially on complex environments like SokobanNewVocab and LargeSokoban. This setting provides enough room for planning while avoiding the chaos of overly long rollouts. Increasing the budget to 7 actions degrades performance, likely

Table 1 | **Effect of Task Diversity on Generalization Performance (%)**. Higher diversity with multiple responses yields the best performance (4 responses per prompt).

Response Per Prompt	SingleSokoban	SokobanNewVocab	FrozenLake
32	21.09%	20.22%	17.97%
16	20.31%	21.48%	19.53%
8	20.31%	19.53%	17.19%
4	20.70%	25.39%	21.48%
2	19.92%	25.00%	12.50%
1	19.53%	22.27%	12.50%

Table 2 | **Performance across environments under different per-turn action budgets (%)**. **5-6 actions per turn yields best performance**, balancing well for effective multi-step planning.

Max Actions / Turn	Sokoban	SokobanNewVocab	LargeSokoban	FrozenLake
1	12.11%	13.67%	1.17%	11.72%
2	16.41%	21.09%	3.52%	18.36%
3	19.53%	19.53%	1.95%	20.88%
4	26.95%	26.95%	5.08%	20.70%
5	28.13%	25.78%	6.25%	21.09%
6	33.59%	31.64%	6.64%	18.36%
7	22.27%	28.52%	3.91%	19.53%

due to noisy transitions and diluted reward feedback.

Frequent rollout updates ensure alignment between optimization targets and current policy behavior. To investigate the effect of rollout freshness, we adopt an *Online-k* rollout strategy, where a single set of rollouts is reused for k consecutive policy updates. A smaller k implies more frequent rollout collection. Notably, *Online-1* corresponds to an fully online setting, with fresh rollouts collected every update iteration. As shown in Figure 7, **agents trained with fresher rollouts (*Online-1*) achieve faster convergence and better generalization** across tasks compared to those with delayed updates (e.g., *Online-5* or *Online-10*). This supports a core design principle for multi-turn RL: **learning is most effective when trajectories reflect the agent’s latest behavior**. Frequent rollout reduces policy-data mismatch and improves optimization stability.

Finding 5: Task diversity, action budget, and rollout frequency affect rollout quality

Diverse task instances enable better policy contrast and generalization across environments. Moderate action budgets provide enough planning space and avoid the noise introduced by overly long sequences. Up-to-date rollouts ensure optimization targets remain aligned with current policy behavior.

4.4. Reasoning Improves Generalization but Fades in Multi-Turn Settings Without Fine-Grained Rewards

We examine **how symbolic reasoning impacts agent generalization**. While reasoning enhances performance in single-turn tasks like Bandit, it **fails to grow or sustain in complex multi-turn environments like Sokoban**. Below, we analyze these effects step-by-step.

Reasoning traces improve generalization in single-turn Bandit tasks. We design a controlled generalization test in symbolic Bandit environments. In the original Bandit setting, the model is trained on the [Teacher, Engineer] arm pair and evaluated on [Librarian, Trader],

Table 3 | **Generalization performance (%) with and without reasoning under StarPO-S.** Disabling reasoning significantly reduces generalization in single-turn Bandit task, but has mixed or marginal effects in multi-turn Sokoban task.

	Train on Bandit		Train on Sokoban			
	Bandit	Bandit-Rev	FrozenLake	LargeSokoban	Sokoban	SokobanNewVocab
StarPO-S	100.00	67.58	19.92	2.34	21.48	18.75
NoThink	81.25	56.25	19.53	2.73	20.73	26.17

Table 4 | **Reasoning length (<think> block length) at different training steps.** Token length generally declines over time, while contextually challenging problems like ReverseBandit require more reasoning than their original counterparts.

Step	Bandit				Sokoban	
	Original	NoThink	Reverse	Reverse NoThink	Original	NoThink
0	66.0	12.7	68.8	12.7	307.1	68.6
100	25.6	12.4	33.7	13.0	104.6	55.3
200	17.6	12.4	30.7	13.0	89.5	60.0

preserving intuitive risk-reward alignments (i.e., Engineer and Trader are high-risk, high-reward). In BanditRev, these associations are inverted, assigning counter-intuitive reward profiles to professions and making reasoning more challenging.

As shown in Table 3, models trained with reasoning traces generalize better in Bandit and even in the counterintuitive BanditRev, suggesting that **reasoning supervision helps internalize symbolic cues beyond memorization**. Despite the added difficulty in BanditRev, models with explicit reasoning consistently outperform those without, as shown in Table 3. This suggests that reasoning traces help the agent **internalize symbolic-reward associations** and **generalize beyond surface-level memorization**, even under semantic-reward misalignment.

In multi-turn tasks, reasoning signals fade as training progresses. In contrast to single-turn settings, we find that reasoning provides limited benefits in multi-turn environments like Sokoban and FrozenLake. Even when the output format includes explicit <think> segments, **removing them (no-think variant) often yields comparable or even better performance**. To understand this degradation, we analyze average response length during training (Table 4, Figure 14) and find that **reasoning traces consistently shrink over time**, suggesting the model is suppressing its own thought process. Interestingly, in the semantically misaligned BanditRev task—where reasoning is essential—traces remain longer, indicating that reasoning is better sustained when context is more challenging.

We presume that reasoning collapse may arise from the **sparse, delayed reward structure in multi-turn tasks**, which often fails to differentiate between coherent reasoning and trial-and-error success. This is supported by examples (Appendix L) where models generate incoherent or hallucinated reasoning yet still receive high rewards. This raises an important challenge: *how can we consistently reinforce useful reasoning when the reward alone may not reflect its quality?* One possible approach is to **decouple action correctness from reasoning quality** using format-based penalties: similar to Shao et al. (2024b), we apply a small penalty to outputs lacking valid <think>–<answer> structure, encouraging structured reasoning. We believe future work may explore finer-grained reward designs such as rewarding partial correctness to reliably reinforce reasoning in long-horizon decision-making.

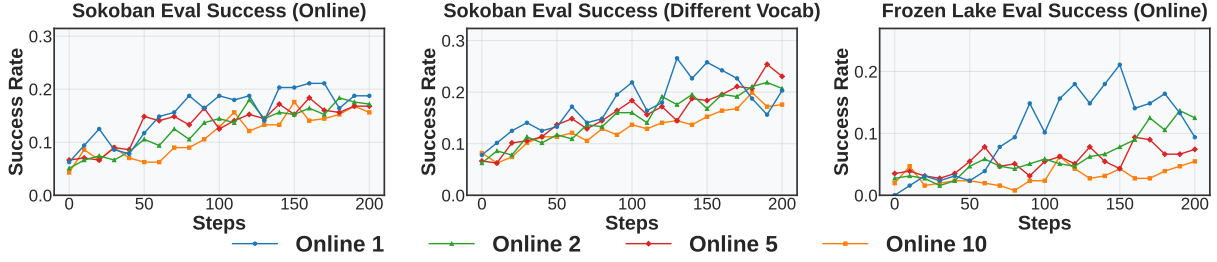


Figure 7 | **Performance under different rollout frequencies (*Online-k*)**. We vary the rollout reuse factor k , where each batch is reused for k policy updates. Lower k (e.g., *Online-1*) means more frequent rollouts. Fresher data improves convergence by **staying aligned with the current policy**.

Finding 6: Reasoning fails to emerge without meticulous reward design

While symbolic reasoning can emerge in simple, single-turn tasks under weak supervision, it fails to persist in multi-turn environments without the reward design explicitly encouraging interpretable intermediate reasoning steps. We observe that even with structured prompts, **reasoning gradually decays during training** if the reward signal focuses only on final outcomes. This suggests that **without meticulous reward shaping, agents may tend to collapse into shortcut behaviors** that bypass reasoning altogether.

5. Related Work

Recent work has explored fine-tuning LLMs for multi-step reasoning through **reinforcement learning (RL)**, and developing agent frameworks to structure decision-making tasks. For reasoning, techniques range from classical PPO (Schulman et al., 2017) and actor-critic methods (Haarnoja et al., 2018) to structured prompting via meta tokens (Goyal et al., 2024; Herel and Mikolov, 2024). Policy variants such as RLOO (Kool et al., 2019), GRPO (DeepSeek-AI et al., 2025) and DAPO (Yu et al., 2025) could stabilize training and improve sample efficiency. Parallel efforts like STaR (Zelikman et al., 2022) and MCTS-based reasoning (Hao et al., 2023a) promote step-by-step reasoning with minimal supervision.

On the agent side, systems have evolved from early **reactive planning** (Xu et al., 2023; Yao et al., 2022b) to **modularized decision pipelines** (Liu et al., 2023; Wu et al., 2023), **multi-agent cooperation** (Li et al., 2023; Wang et al., 2024a), and **embodied interaction** (Li et al., 2025; Lin et al., 2024b). Benchmarks like **Sokoban** (Junghanns and Schaeffer, 2001), **FrozenLake** (Dell’Aversana, 2021), and **WebShop** (Yao et al., 2022a) provide controlled testbeds for evaluating reasoning under different dynamics. Our work builds on these advances, aiming to unify RL-based reasoning with structured agent training across symbolic and language-centric tasks. We summarize more related works in Appendix B.

6. Conclusions and Limitations

We present RAGEN, a general-purpose system for training language agents with reinforcement learning in multi-turn, stochastic environments. Built upon the StarPO framework, RAGEN enables reasoning-guided trajectory optimization and reveals new challenges unique to agent training—such as **gradient collapse, rollout drift, and reasoning degradation**. Through extensive experiments, we identify key design principles for stabilizing training, including **rollout filtering, gradient shaping, and reward-aware reasoning supervision**. These insights provide a foundation

for building more robust and generalizable LLM agents. Our framework offers a scalable platform for studying autonomous language agents across domains such as symbolic reasoning and web browsing. Limitations of our work include the focus on relatively small-scale tasks, the omission of established RL practices like replay buffers, and the absence of multimodal tasks—which we leave for future work.

Acknowledgements

We thank the DeepSeek team for providing the DeepSeek-R1 model and early conceptual inspirations. We are grateful to the veRL team for their infrastructure support, and to the TinyZero team for their discoveries that informed our initial exploration. We would like to appreciate insightful discussions with Han Liu, Xinyu Xing, Monica Lam, Li Erran Li, John Schulman, Akari Asai, Eiso Kant, Lu Lu, Runxin Xu, Zhihan Liu, Huajian Xin, Zijun Liu, Weiye Liu, Weimin Wu, Yibo Wen, Jiarui Liu, Lorenzo Xiao, Ishan Mukherjee, Anabella Isaro, Haosen Sun, How-Yeh Wan, Lester Xue, Matthew Khoriaty, Haoxiang Sun, Jiajun Liu.

References

- G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym, 2016. URL <https://arxiv.org/abs/1606.01540>.
- L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *arXiv preprint arXiv:2106.01345*, 2021.
- W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C. Qian, C.-M. Chan, Y. Qin, Y. Lu, R. Xie, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *arXiv preprint arXiv:2308.10848*, 2(4):6, 2023.
- P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences, 2023. URL <https://arxiv.org/abs/1706.03741>.
- DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. *CoRR*, abs/2401.02954, 2024. URL <https://doi.org/10.48550/arXiv.2401.02954>.
- DeepSeek-AI, Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma, W. Zeng, X. Bi, Z. Gu, H. Xu, D. Dai, K. Dong, L. Zhang, Y. Piao, Z. Gou, Z. Xie, Z. Hao, B. Wang, J. Song, D. Chen, X. Xie, K. Guan, Y. You, A. Liu, Q. Du, W. Gao, X. Lu, Q. Chen, Y. Wang, C. Deng, J. Li, C. Zhao, C. Ruan, F. Luo, and W. Liang. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence, 2024. URL <https://arxiv.org/abs/2406.11931>.
- DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S.

- Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- P. Dell’Aversana. The frozen lake problem. an example of optimization policy, 12 2021.
- Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- L. Feng, Z. Xue, T. Liu, and B. An. Group-in-group policy optimization for llm agent training, 2025. URL <https://arxiv.org/abs/2505.10978>.
- Z. Gao, W. Zhan, J. D. Chang, G. Swamy, K. Brantley, J. D. Lee, and W. Sun. Regressing the relative future: Efficient policy optimization for multi-turn rlhf, 2024. URL <https://arxiv.org/abs/2410.04612>.
- S. Goyal, Z. Ji, A. S. Rawat, A. K. Menon, S. Kumar, and V. Nagarajan. Think before you speak: Training language models with pause tokens, 2024. URL <https://arxiv.org/abs/2310.02226>.
- T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018. URL <https://arxiv.org/abs/1801.01290>.
- S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023a.
- S. Hao, T. Liu, Z. Wang, and Z. Hu. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in neural information processing systems*, 36: 45870–45894, 2023b.
- A. Havrilla, Y. Du, S. C. Raparthy, C. Nalmpantis, J. Dwivedi-Yu, M. Zhuravinskyi, E. Hambro, S. Sukhbaatar, and R. Raileanu. Teaching large language models to reason with reinforcement learning, 2024. URL <https://arxiv.org/abs/2403.04642>.
- C. He, B. Zou, X. Li, J. Chen, and H. M. Junliang Xing. Enhancing llm reasoning with multi-path collaborative reactive and reflection agents, 2025. URL <https://arxiv.org/abs/2501.00430>.
- D. Herel and T. Mikolov. Thinking tokens for language modeling, 2024. URL <https://arxiv.org/abs/2405.08644>.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.

- J. Hu, Y. Zhang, Q. Han, D. Jiang, X. Zhang, and H.-Y. Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model, 2025a. URL <https://arxiv.org/abs/2503.24290>.
- L. Hu, M. Huo, Y. Zhang, H. Yu, E. P. Xing, I. Stoica, T. Rosing, H. Jin, and H. Zhang. Imgame-bench: How good are llms at playing games?, 2025b. URL <https://arxiv.org/abs/2505.15146>.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2024.
- B. Jin, J. Yoon, P. Kargupta, S. O. Arik, and J. Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents, 2025a. URL <https://arxiv.org/abs/2505.15117>.
- B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning, 2025b. URL <https://arxiv.org/abs/2503.09516>.
- A. Junghanns and J. Schaeffer. Sokoban: Enhancing general single-agent search methods using domain knowledge. *Artificial Intelligence*, 129(1):219–251, 2001. ISSN 0004-3702. doi: [https://doi.org/10.1016/S0004-3702\(01\)00109-6](https://doi.org/10.1016/S0004-3702(01)00109-6). URL <https://www.sciencedirect.com/science/article/pii/S0004370201001096>.
- W. Kool, H. van Hoof, and M. Welling. Buy 4 REINFORCE samples, get a baseline for free!, 2019. URL <https://openreview.net/forum?id=r1lgTGL5DE>.
- A. Kumar, V. Zhuang, R. Agarwal, Y. Su, J. D. Co-Reyes, A. Singh, K. Baumli, S. Iqbal, C. Bishop, R. Roelofs, L. M. Zhang, K. McKinney, D. Shrivastava, C. Paduraru, G. Tucker, D. Precup, F. Behbahani, and A. Faust. Training language models to self-correct via reinforcement learning, 2024. URL <https://arxiv.org/abs/2409.12917>.
- G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.
- M. Li, S. Zhao, Q. Wang, K. Wang, Y. Zhou, S. Srivastava, C. Gokmen, T. Lee, L. E. Li, R. Zhang, W. Liu, P. Liang, L. Fei-Fei, J. Mao, and J. Wu. Embodied agent interface: Benchmarking llms for embodied decision making, 2025. URL <https://arxiv.org/abs/2410.07166>.
- H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- B. Y. Lin, Y. Fu, K. Yang, F. Brahman, S. Huang, C. Bhagavatula, P. Ammanabrolu, Y. Choi, and X. Ren. Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks. *Advances in Neural Information Processing Systems*, 36, 2024a.
- J. Lin, H. Zhao, A. Zhang, Y. Wu, H. Ping, and Q. Chen. Agentsims: An open-source sandbox for large language model evaluation, 2023. URL <https://arxiv.org/abs/2308.04026>.
- J. Lin, H. Gao, X. Feng, R. Xu, C. Wang, M. Zhang, L. Guo, and S. Xu. Advances in embodied navigation using large language models: A survey, 2024b. URL <https://arxiv.org/abs/2311.00530>.

- B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- Z. Liu, H. Hu, S. Zhang, H. Guo, S. Ke, B. Liu, and Z. Wang. Reason for future, act for now: A principled framework for autonomous llm agents with provable sample efficiency, 2024. URL <https://arxiv.org/abs/2309.17382>.
- Z. Liu, C. Chen, W. Li, P. Qi, C. D. Tianyu Pang, W. S. Lee, and M. Lin. Understanding rl-zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.
- M. Nguyen, A. Baker, C. Neo, A. Roush, A. Kirsch, and R. Shwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent llm outputs, 2024. URL <https://arxiv.org/abs/2407.01082>.
- OpenAI. Introducing ChatGPT o1, 2024. URL <https://openai.com/o1/>. Accessed: 2025-02-15.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- J. Pan, J. Zhang, X. Wang, L. Yuan, H. Peng, and A. Suhr. Tinyzero. <https://github.com/Jiayi-Pan/TinyZero>, 2025. Accessed: 2025-01-24.
- J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.
- J. Pfau, W. Merrill, and S. R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models, 2024. URL <https://arxiv.org/abs/2404.15758>.
- Z. Qi, X. Liu, I. L. Iong, H. Lai, X. Sun, W. Zhao, Y. Yang, X. Yang, J. Sun, S. Yao, T. Zhang, W. Xu, J. Tang, and Y. Dong. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning, 2025. URL <https://arxiv.org/abs/2411.02337>.
- Y. Qin, Y. Ye, J. Fang, H. Wang, S. Liang, S. Tian, J. Zhang, J. Li, Y. Li, S. Huang, W. Zhong, K. Li, J. Yang, Y. Miao, W. Lin, L. Liu, X. Jiang, Q. Ma, J. Li, X. Xiao, K. Cai, C. Li, Y. Zheng, C. Jin, C. Li, X. Zhou, M. Wang, H. Chen, Z. Li, H. Yang, H. Liu, F. Lin, T. Peng, X. Liu, and G. Shi. Ui-tars: Pioneering automated gui interaction with native agents, 2025. URL <https://arxiv.org/abs/2501.12326>.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018. URL <https://arxiv.org/abs/1506.02438>.
- B. Settles. Active learning literature survey. 2009.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024a. URL <https://arxiv.org/abs/2402.03300>.

- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024b.
- Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2024.
- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. J. Anderson, and Y. Gal. Ai models collapse when trained on recursively generated data. *Nat.*, 631(8022):755–759, July 2024. URL <https://doi.org/10.1038/s41586-024-07566-y>.
- Y. Song, D. Yin, X. Yue, J. Huang, S. Li, and B. Y. Lin. Trial and error: Exploration-based trajectory optimization for llm agents, 2024.
- C. J. Wang, D. Lee, C. Menghini, J. Mols, J. Doughty, A. Khoja, J. Lynch, S. Hendryx, S. Yue, and D. Hendrycks. Enigmaeval: A benchmark of long multimodal reasoning challenges, 2025. URL <https://arxiv.org/abs/2502.08859>.
- K. Wang*, P. Zhang*, Z. Wang*, Q. Wang*, Y. Gao*, L. Li*, Z. Yang, C. Wan, H. Chen, Y. Lu, and M. Li. Vagen: Training vlm agents with multi-turn reinforcement learning, 2025. URL <https://github.com/RAGEN-AI/VAGEN>.
- Q. Wang, Z. Wang, Y. Su, H. Tong, and Y. Song. Rethinking the bounds of llm reasoning: Are multi-agent discussions the key? *arXiv preprint arXiv:2402.18272*, 2024a.
- X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji. Mint: Evaluating llms in multi-turn interaction with tools and language feedback, 2024b. URL <https://arxiv.org/abs/2309.10691>.
- Z. Wei, W. Yao, Y. Liu, W. Zhang, Q. Lu, L. Qiu, C. Yu, P. Xu, C. Zhang, B. Yin, H. Yun, and L. Li. Webagent-r1: Training web agents via end-to-end multi-turn reinforcement learning, 2025. URL <https://arxiv.org/abs/2505.16421>.
- Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- J. Xi, Y. He, J. Yang, Y. Dai, and J. Chai. Teaching embodied reinforcement learning agents: Informativeness and diversity of language use, 2024. URL <https://arxiv.org/abs/2410.24218>.
- T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, et al. Openagents: An open platform for language agents in the wild. *arXiv preprint arXiv:2310.10634*, 2023.
- B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

- S. Yao, H. Chen, J. Yang, and K. Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022a.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022b.
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, W. Dai, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022. URL <https://arxiv.org/abs/2203.14465>.
- S. Zeng, Q. Wei, W. Brown, O. Frunza, Y. Nevmyvaka, and M. Hong. Reinforcing multi-turn reasoning in llm agents via turn-level credit assignment, 2025a. URL <https://arxiv.org/abs/2505.11821>.
- W. Zeng, Y. Huang, Q. Liu, W. Liu, K. He, Z. Ma, and J. He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025b. URL <https://arxiv.org/abs/2503.18892>.
- W. Zeng, Y. Huang, W. Liu, K. He, Q. Liu, Z. Ma, and J. He. 7b model and 8k examples: Emerging reasoning with reinforcement learning is both effective and efficient. <https://hkust-nlp.notion.site/simplerl-reason>, 2025c. Notion Blog.
- Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, and J. Lin. The lessons of developing process reward models in mathematical reasoning, 2025. URL <https://arxiv.org/abs/2501.07301>.
- Y. Zhou, A. Zanette, J. Pan, S. Levine, and A. Kumar. Archer: Training language model agents via hierarchical multi-turn rl, 2024. URL <https://arxiv.org/abs/2402.19446>.
- Y. Zhuang, X. Chen, T. Yu, S. Mitra, V. Bursztyin, R. A. Rossi, S. Sarkhel, and C. Zhang. Toolchain*: Efficient action space navigation in large language models with a* search. *arXiv preprint arXiv:2310.13227*, 2023.

A. Background of Reinforcement Learning

Reinforcement learning (RL) enables foundation models to learn through interaction and reward signals. The general RL objective is:

$$J(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta(\cdot|s)} [R(s, a)], \quad (8)$$

where π_θ is the policy, s is the input prompt, a is the response, and $R(s, a)$ is the reward function evaluating response quality.

Common approaches use reward modeling and policy optimization for RL. Proximal Policy Optimization (PPO) (Schulman et al., 2017) stabilizes training through probability ratio clipping and advantage estimation. The probability ratio is defined as:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (9)$$

The PPO objective uses this ratio with clipping:

$$J_{PPO}(\theta) = \mathbb{E}_t [\min(\rho_i A_i, \hat{\rho}_i A_i) - \beta D_{KL}], \quad (10)$$

with probability ratio $\rho_i = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}$ and clipped ratio $\hat{\rho}_i = \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon)$.

For advantage estimation, Generalized Advantage Estimation (GAE) (Schulman et al., 2018) computes:

$$A_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (11)$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD error, and (γ, λ) control the bias-variance tradeoff.

Recently, DeepSeek-R1-Zero (DeepSeek-AI, 2024) implements this paradigm through Group Relative Policy Optimization (GRPO), sampling G outputs $\{o_i\}$ [consisting of reasoning and actions] for each prompt and optimizes:

$$J_{GRPO}(\theta) = \mathbb{E}_{q, \{o_i\}} [J_{group}(\theta)], \quad (12)$$

where:

$$J_{group}(\theta) = \frac{1}{G} \sum_{i=1}^G \min(\rho_i A_i, \hat{\rho}_i A_i) - \beta D_{KL}, \quad (13)$$

while mostly similar to Eq. 3, the GRPO advantage is neural-model free and calculated as:

$$A_i = \frac{r_i - \text{mean}(\{r_j\})}{\text{std}(\{r_j\})}. \quad (14)$$

Using rule-based rewards r_i , this pure RL approach demonstrates emergent reasoning behaviors.

B. Extended Related Work

Reinforcement Learning for Reasoning in LLMs. Reinforcement learning (RL) on LLMs (Chen et al., 2021; Christiano et al., 2023; Havrilla et al., 2024; Ouyang et al., 2022) has significantly improved LLMs’ reasoning capabilities. Notable approaches include the use of Proximal Policy Optimization Algorithms (PPO) (Schulman et al., 2017) which maintains training stability while enhancing performance by clipping policy updates, Group Relative Policy Optimization (GRPO)

(DeepSeek-AI et al., 2025) for enhancing the ability of systematic problem-solving, actor-critic methods like SAC (Haarnoja et al., 2018) and ArCHer (Zhou et al., 2024) that leverages an critic to promote robust exploration and stability, and meta tokens (Goyal et al., 2024; Herel and Mikolov, 2024; Pfau et al., 2024) for structured thinking. Other significant developments include Process Reward Model (PRM) (Lightman et al., 2023; Zhang et al., 2025) and Monte Carlo Tree Search (MCTS) based approaches (Hao et al., 2023a) for systematic problem-solving. On the other hand, recent advances in LLM reasoning have explored techniques to enable models to generate intermediate chain-of-thought rationales. In particular, STaR (Zelikman et al., 2022) iteratively leverages a small set of rationale examples along with a large dataset without rationales. Recent work like SimpleRL-Zoo (Zeng et al., 2025b) DAPO (Yu et al., 2025), RLOO (Kool et al., 2019), Dr. GRPO (Liu et al., 2025), and Open Reasoner Zero (Hu et al., 2025a) all demonstrate that minimalist, reproducible RL techniques—featuring decoupled clipping, unbiased optimization, and simple reward schemes—can significantly enhance LLM reasoning performance.

Existing agent frameworks. LLM-based agent architectures have evolved from early reasoning-action frameworks (Lin et al., 2024a; Shinn et al., 2024; Xu et al., 2023; Yao et al., 2022b) to structured approaches (Hao et al., 2023a; Liu et al., 2023, 2024; Zeng et al., 2025a). Multi-agent systems (Chen et al., 2023; Du et al., 2023; Li et al., 2023; Wang et al., 2024a) are designed for tasks with more complex interactions. Widely used platforms such as OpenAI Gym (Brockman et al., 2016) and specialized environments including Sokoban (Junghanns and Schaeffer, 2001), FrozenLake (Dell’Aversana, 2021), and Webshop (Yao et al., 2022a) provide diverse testbeds for evaluating these agents. Moreover, general-purpose systems (Hao et al., 2023b; Shen et al., 2024; Wu et al., 2023; Xie et al., 2023; Zhuang et al., 2023) have enabled broad applications ranging from web navigation and search (Jin et al., 2025a,b; Qi et al., 2025; Wei et al., 2025), coding copilot (DeepSeek-AI et al., 2024; Jimenez et al., 2024; Wang et al., 2024b) to GUI (Qin et al., 2025; Yao et al., 2022a), Game (Hu et al., 2025b) and embodied tasks (Feng et al., 2025; Li et al., 2025; Lin et al., 2024b; Xi et al., 2024). Social interaction capabilities have been advanced through Generative Agents and AgentSims (Lin et al., 2023; Park et al., 2023). Challenges persist in architectural complexity and self-correction (He et al., 2025), especially for diverse, multi-step reasoning tasks (Nguyen et al., 2024; Song et al., 2024; Wang et al., 2025).

C. Detailed Experimental Settings

C.1. Environments and Tasks

We construct a **diverse four-environment testbed** to evaluate LLM agents across key axes of decision-making complexity. Three environments—Bandit, Sokoban, and Frozen Lake—are symbolic, synthetic, and fully controllable, supporting clean analysis of RL learning from scratch. They are deliberately minimal and decoupled from real-world priors; even large models like GPT-4o perform poorly without training, highlighting the need for grounded policy learning. Complementarily, we include WebShop, a realistic multi-turn task involving natural language grounding and web navigation in a semi-structured interface. Together, the four environments enable systematic study of reasoning, training stability, and generalization in agentic LLMs across symbolic and open-domain settings.

Each environment stresses a distinct capability: Bandits tests reasoning under uncertainty, Sokoban emphasizes irreversible long-horizon planning, Frozen Lake involves stochastic transitions, and WebShop requires language understanding and goal-directed interaction.

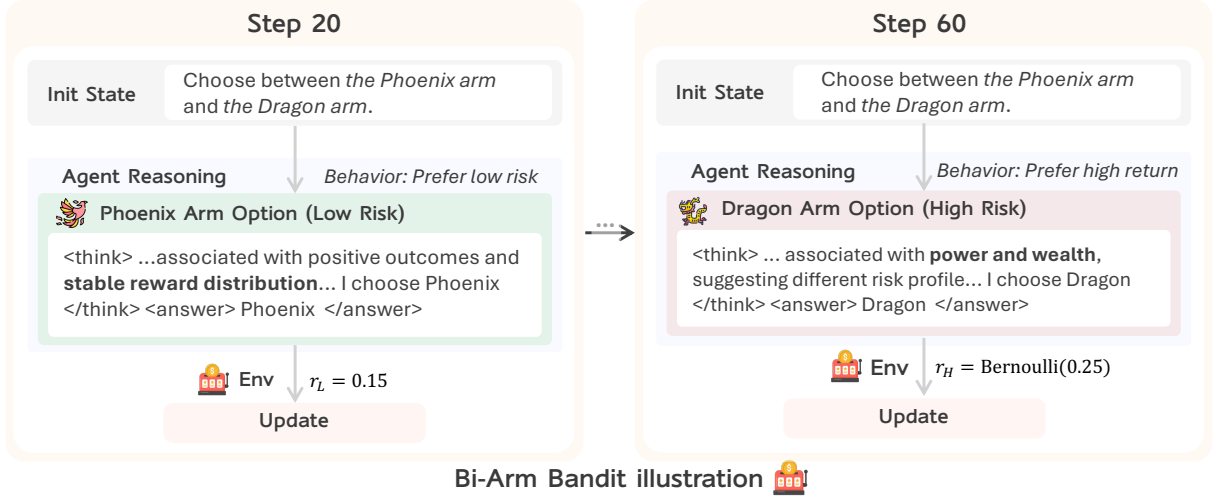


Figure 8 | **Bi-Arm Bandits environment.** The agent chooses between a low-risk arm (Phoenix) and a high-risk yet high-reward arm (Dragon), each linked to symbolic semantics. The agent learns to choose stable reward at early stages and reasons to pursue maximal expected reward and shift toward strategic risk-taking.

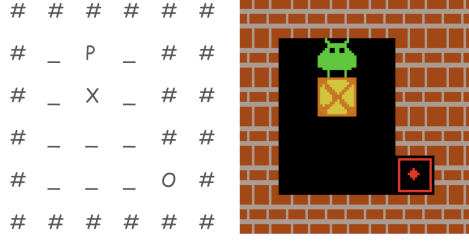
Bi-Arm Bandits. We design this environment to evaluate whether agents can **form risk-sensitive hypotheses and revise them based on training**. At each step, the agent must choose between two semantically symbolic options—e.g., “Dragon” vs. “Phoenix”—each linked to a fixed reward distribution (Figure 8). The low-risk arm always returns a reward of 0.15, while the high-risk arm samples from *Bernoulli*(0.25): higher variance, higher expected return.

Importantly, the low-risk arm wins more often per trial, even though the high-risk arm is better in expectation. This is designed to test reasoning: without inductive bias, models may prefer the lo-arm due to its more frequent success, but a reasoning agent must learn to associate symbolic cues (e.g., “Dragon”) with underlying reward statistics, override misleading short-term signals, and “justify” high-risk choices based on long-term expected return. We further test this by reversing the symbolic labels to probe agent’s reasoning under opposed reward systems.

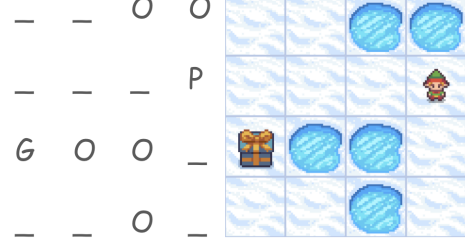
Sokoban. We use the puzzle Sokoban (Figure 9a) to study multi-turn agent interaction. The agent must push a box to the goal in a grid within constrained steps. Unlike standard navigation, Sokoban is irreversible: boxes can only be pushed, not pulled back, which requires the agent to reason ahead to avoid dead-ends. The reward signal encourages efficiency and accuracy: +1 for each box on target, −1 for off-target boxes, +10 upon task completion, and −0.1 per action.

Frozen Lake. This environment (Figure 9b) combines long-horizon decision-making with stochastic transitions. The agent navigates a grid with slippery tiles; each action succeeds with probability 1/3 and deviates perpendicularly with probability 2/3. The agent should reach the goal without falling into holes. Rewards are sparse: successful trials receive a reward of +1, with all others 0.

WebShop. To complement the symbolic environments, we include WebShop Yao et al. (2022a), a multi-turn web-based shopping task that tests agents’ ability to ground natural language queries, navigate semi-structured interfaces, and retrieve goal-relevant information. The agent must select a product that matches a user’s request by issuing search queries, clicking links, and reading product descriptions—introducing realistic language grounding and action space challenges absent in purely symbolic settings.



(a) Sokoban Environment. Actions are irreversible and the boxes cannot be pulled back which needs foresight and careful planning.



(b) FrozenLake Environment. Each agent move may randomly slip to a side direction, requiring the agent to adapt under uncertainty.

Figure 9 | **Sokoban and Frozen Lake environments.** For each environment, the left shows the agent-observed text rendering; the right is a visual illustration. (a) Sokoban is a deterministic multi-turn puzzle where the agent pushes boxes onto targets. (b) Frozen Lake combines multi-turn reasoning and stochasticity where the agent needs to reach the gift to succeed.

C.2. Training and Evaluation Settings

We conduct our experiments using Qwen2.5-0.5B-Instruct (Yang et al., 2024), trained via the StarPO variants with a maximum of 200 rollout-update iterations on NVIDIA H100/A100 GPUs leveraging the verL* repository. Considering the extremely long training time of WebShop due to its long-context nature, we train on WebShop for 100 steps. Each rollout consists of $K = 16$ trajectories per environment group, based on prompt size $P = 8$ and maximum 5 interaction turns per episode. Agents are allowed up to 5 actions per turn and 10 actions per episode. The update batch size is $E = 32$, with mini-batch size 4 per GPU. Policy optimization uses GAE with ($\gamma = 1.0, \lambda = 1.0$) and Adam with $(\beta_1, \beta_2) = (0.9, 0.999)$. We use entropy regularization ($\beta = 0.001$) For experiments with vanilla StarPO we use a KL coefficient of 0.001, using the k1 estimation[†]. without KL loss term during training, following (Yu et al., 2025), and track KL post-hoc. We impose a format penalty of -0.1 if the agent fails to output valid structured responses (e.g., missing `<think>` or `<answer>` tags), encouraging adherence to response conventions. To accelerate rollout generation, we disable `enforce_eager` and retain the computation graph across prefill and sampling in vLLM. We utilize Fully Sharded Data Parallel (FSDP) training strategy for multi-GPU experiments. For distributed training, we employ Ray as the multi-processing backend with XFORMERS attention implementation.

For evaluation, we choose a fixed 256 input prompts per environment and decode using temperature $T=0.5$, sampling stochastically to better capture robustness in agent behaviors. Episode truncation occurs after 5 turns or 10 total actions.

C.3. Evaluation Metrics

To track agent learning dynamics and detect training instabilities, we monitor the following metrics throughout training. Except for the success rate, which is evaluated on a fixed validation set, all metrics are computed over validation instances.

- **Average Success Rate.** Measures task completion accuracy on a fixed set of validation prompts. An episode is considered successful if the agent solves the task (e.g., pulling the

^{*}<https://github.com/volcengine/verl>

[†]<http://joschu.net/blog/kl-approx.html>

high-reward arm in Bandit, pushing all boxes to targets in Sokoban, reaching the goal in Frozen Lake, and a successful purchase in WebShop).

- **Rollout Entropy.** Computes the average token-level entropy of sampled responses, capturing the exploration level and policy uncertainty. A sharp entropy drop may indicate premature policy convergence or collapse.
- **In-Group Reward Variance.** Measures reward standard deviation across rollouts sampled from the same prompt group. High in-group variance reflects diverse behaviors and learning potential; a sudden collapse indicates reward homogenization and policy stagnation.
- **Total Response Length.** Average number of tokens generated per rollout, measuring the verbosity and reasoning depth of the agent. Fluctuations in length may signal changes in planning style or confidence.
- **Gradient Norm.** ℓ_2 norm of the policy gradient vector, used as a proxy for training stability. Spikes often correlate with phase transitions in policy behavior or unstable reward signals.

These metrics provide complementary views of policy quality, update dynamics, and reasoning behavior, helping diagnose when and why agent training succeeds or fails.

D. Results on Larger Models and Various Optimization Algorithms

We extend all our evaluation to 3B/7B/72B scale model and explore the effects of various algorithm choices like KL removal and asymmetric clipping, together with turn-aware optimization techniques including Generalized Advantage Estimation (GAE) and response masking.

Scaling Effects. We extend our trained model to 3B / 7B to evaluate scaling effects of RL training. Results can be found in Figure 10. Note that due to the extremely long context length of WebShop could due to OOM Error within 4xH100 for 7B models, we only report 3B performance in WebShop task. On **Bandit** and **WebShop**, the larger model demonstrates significantly stronger performance compared to smaller models. However, the improvements are marginal on **Sokoban** and **FrozenLake**. We attribute this discrepancy to the nature of the environments: Sokoban and FrozenLake are symbolic, grid-based tasks with minimal overlap with pretraining data, limiting the model’s ability to leverage language priors. In contrast, Bandit and WebShop involve natural language interactions, allowing pretrained models to more effectively exploit linguistic patterns for policy learning, even in the absence of explicit environment dynamics. This can be further validated through cases in Figure 16, 17, 18, 19, where semantic-rich tasks like Bandit and WebShop presents significantly more diverse reasoning patterns and better benefit from scale.

Frontier Model Performance. To contextualize our small model’s performance, we evaluate two large foundation models—**GPT-4o** and **Qwen2.5-72B-Instruct**—on SimpleSokoban and FrozenLake in a zero-shot setting. Both models are prompted with task instructions and example formats, without any fine-tuning or in-context trajectory rollouts. See Table 5

While GPT-4o and Qwen2.5-72B achieve 19–28% success rates in Sokoban and FrozenLake without any task-specific adaptation, our 0.5B model reaches **20.70%** and **21.48%** respectively after training from scratch. This result is notable given that our model has over **100× fewer parameters**. It highlights that even under strict resource constraints, careful rollout construction

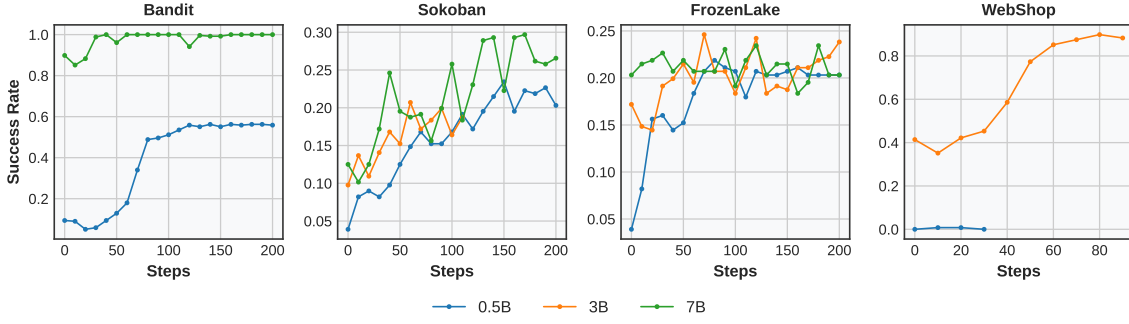


Figure 10 | **Scaling effect across environments.** The larger model outperforms smaller models on Bandit and WebShop, which benefit from language priors, but shows limited gains on symbolic, grid-based environments like Sokoban and FrozenLake.

Model	Sokoban (%)	FrozenLake (%)	Response Length	Effective Action (%)
GPT-4o	27.73	26.56	190.3 / 132.1	70.0 / 88.7
Qwen2.5-72B	19.53	23.83	179.4 / 107.3	67.8 / 87.3
0.5B (ours)	20.70	21.48	89.5 / 110.6	79.2 / 80.7

Table 5 | **Zero-shot vs. trained performance.** Our 0.5B model, trained with only 4 responses per prompt, achieves performance comparable to that of large foundation models without any fine-tuning. Response length and effectiveness values for GPT-4o / Qwen are reported per environment.

and policy optimization (see Section 4.3) can match the generalization ability of significantly larger models.

Gradient Shaping. We evaluate the effectiveness of KL Term Removal and Clip-Higher Yu et al. (2025), finding them useful by simply extending from single-turn static tasks to agent tasks:

- **KL Term Removal:** We eliminate the KL divergence penalty from PPO’s objective, relying only on policy loss and entropy bonus for gradient updates. It removes the constraint to stay close to the initial model distribution and encourage the model to explore.
- **Clip-Higher (Asymmetric Clipping):** We decouple the PPO clipping range by using a higher upper bound ($\epsilon_{\text{high}} = 0.28$) than the lower bound ($\epsilon_{\text{low}} = 0.2$). It allows the model to learn more aggressively from high-reward rollouts for more effective training.

As shown in Figure 11, both methods boost the success rate and extend stable training phases, showing how multi-turn RL benefits from more flexible gradient shaping.

Response Masking and Bi-Level GAE. Following the turn-aware optimization strategy proposed in Wang* et al. (2025), we evaluate the effect of response masking and bi-level GAE on a 0.5B model. As shown in Figure 12, both techniques contribute to improved performance in multi-turn RL tasks, which highlight the promise of turn-aware RL training algorithms for stabilizing and enhancing language agent training.

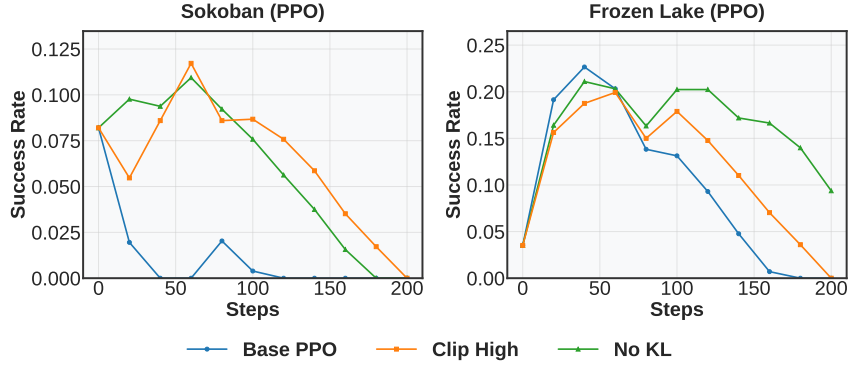


Figure 11 | **Effect of KL removal and asymmetric clipping on PPO stability.** The two designs both improve peak performance and delay collapse in multi-turn RL.

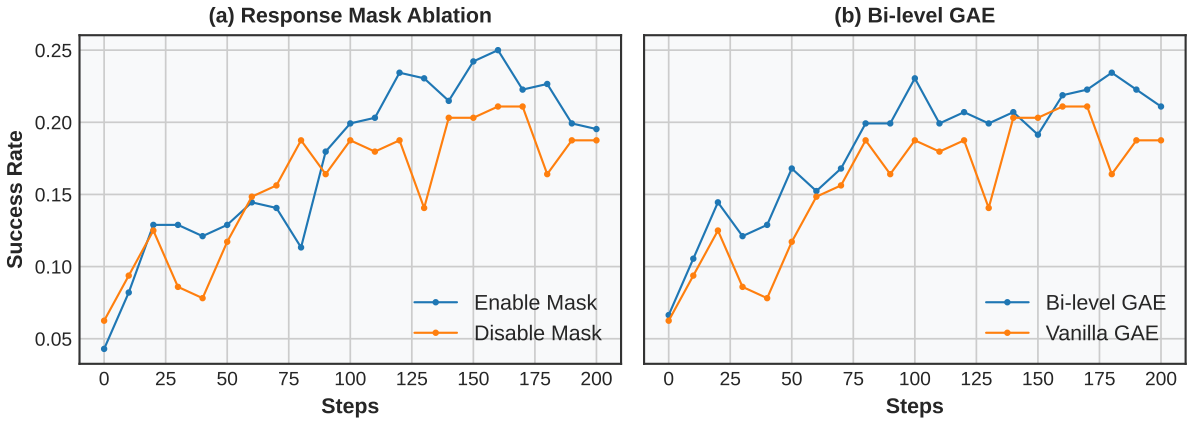


Figure 12 | **Ablation on turn-aware optimization strategies on Sokoban Task.** Both response masking and bi-level GAE improve multi-turn RL performance.

E. When Does Uncertainty-Based Filtering Help?

We hypothesize that the effectiveness of StarPO-S largely depends on the variance of rollout rewards within each environment. In settings where the task is either too easy or too difficult, the generated trajectories tend to have very low intra-group variance—meaning the model is overconfident or uniformly poor across samples. In such cases, standard StarPO may propagate misleading gradients, while StarPO-S helps by filtering out low-confidence rollouts. Conversely, in open-ended or more diverse environments (like WebShop), the rollout variance tends to be naturally high, which reduces the marginal benefit of StarPO-S filtering.

Figure 13 supports this intuition. The top row shows the success rates of StarPO and StarPO-S across four environments, while the bottom row shows the evolution of `in_group_std` and `chosen_in_group_std` over training. In Bandit, Sokoban, and FrozenLake, StarPO-S consistently outperforms StarPO, with the gap widening as the rollout variance drops. In WebShop, however, the high and stable variance suggests more diversity in generated responses, making the filtering effect of StarPO-S less critical—explaining the smaller performance gap.

These results indicate that StarPO-S is most beneficial when environments exhibit low rollout uncertainty, providing a simple diagnostic for when to apply it.

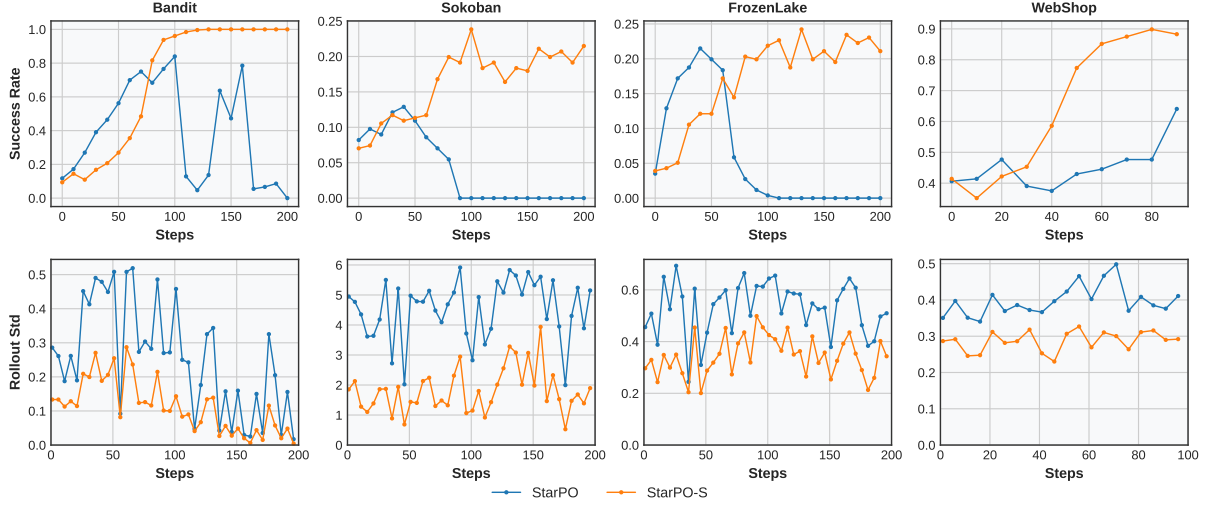


Figure 13 | **Comparison of success rate (top) and rollout variance (bottom).** StarPO-S basically improve training stability on the environments having extremely easy or difficult problems like Sokoban and Frozen Lake. This would ead to instances with small rollout Stds which can be easily filtered out through StarPO-S to make training more stable. On Tasks like WebShop, the rollout Std is already consistently high, and StarPO itself already can achieve good performance.

F. Case Study: The Emergence of Echo Trap with RL

We show cases that demonstrates the Echo Trap in RL training. As shown in Table 6, the top rows (Step 0) display varied hypotheses about Dragon and Phoenix, while the bottom rows (Step 150) converge to near-identical phrasing focused on “choosing Dragon” without justification.

G. Comparing Agent RL with Supervised Fine-Tuning

Apart from StarPO for RL training, we also employ Supervised Fine-tuning (SFT) as another agent training approach, evaluating it on the Sokoban and Frozen Lake task. We employ LoRA with a rank of 64 and an alpha value of 32, targeting all linear layers in the model. The SFT process uses a learning rate of $1e-4$ with a training batch size of 128. We generate ground-truth trajectory data through breadth-first search (BFS), setting a maximum depth of 100 to create 1,000 training samples and 100 test samples. For SFT, we structure the multi-turn interaction as a conversational format. At each turn, the model must generate the next action from the ground-truth trajectory, encapsulating its response within `<answer>` `</answer>` tags to maintain format consistency.

We analyze the comparative performance of SFT against our stable RL baseline StarPO-S. SFT achieves 74.6% and 23% performance on Sokoban and Frozen Lake, respectively, Compared to the 20.3% and 21.8% performance with StarPO-S. The results indicate that SFT demonstrates superior performance to RL approaches. We draw conclusions from the results that although rule-based RL show promise for agent tasks, there is still a need to build more scalable and effective agent RL algorithms to achieve human-comparable performance with solely model self-evolution.

Step	Model Output
0	Dragon is often associated with patience, strength, and reliability, while Phoenix is associated with speed, agility, and creativity.
0	The symbolic meaning of Dragon and Phoenix’s names suggests that they may have different reward distributions... Therefore, I think Phoenix is more likely to give higher rewards on average.
150	My initial intention is to pick arm Dragon. My first action is to go for arm Dragon.
150	My initial intention is to pick arm Dragon, so my first action is to go for arm Dragon.
150	My initial intention is to choose Arm Dragon. After analyzing both arms, my intention was to choose Arm Dragon.

Table 6 | **Examples of reasoning patterns in the Bandit task.** Top rows show diverse reasoning from model before training; bottom rows show repetitive and collapsed reasoning after RL training.

H. Efficient Training with Low-Rank Adaptation (LoRA)

Motivation. While the main body of the paper reports results obtained by full-parameter fine-tuning, in practice such a setting may be prohibitive when scaling to larger models or longer-horizon tasks. We therefore implement a parameter-efficient variant of RAGEN based on Low-Rank Adaptation (Hu et al., 2021).[‡]

Performance parity. Despite updating only a fraction of the model parameters, LoRA reaches a validation success rate comparable to that achieved by full fine-tuning of the entire network for the SimpleSokoban task, achieving approximately a 0.2% success rate on the validation set.

Resource savings. We compare the hardware footprint of LoRA with full fine-tuning. Across an 80-minute training horizon we measure:

- **GPU memory.** LoRA stabilizes at $\approx 23\%$ of device memory versus $\approx 48\%$ for full updates, cutting the peak allocation by $>50\%$.
- **GPU utilization.** Average GPU utilization drops from $\sim 34\%$ to $\sim 14\%$.
- **Power consumption.** Mean power draw decreases from $\sim 22\%$ to $\sim 12\%$, a $\approx 45\%$ reduction.

Take-aways. Parameter-efficient fine-tuning provides a practically viable alternative for RAGEN: it attains comparable policy quality while more than halving memory, compute, and power demands. Consequently, future work that scales StarPO to larger backbones or longer contexts can adopt LoRA (or other adapter-based methods) as the default optimization strategy without re-engineering the training loop.

I. PPO Failure Mode in Frozen Lake

Among the three evaluated environments, we observe an interesting divergence on Frozen Lake: PPO tends to collapse earlier or converge less stably than GRPO. This contrasts with the general trend where PPO demonstrates better performance, prompting further analysis.

One possible explanation lies in the environment’s long-horizon stochasticity. In Frozen

[‡]We set rank $r=64$, $\alpha=64$, and inject adapters into all linear projections of the transformer blocks. We also increased learning rate by $10\times$ for both actor and critic.

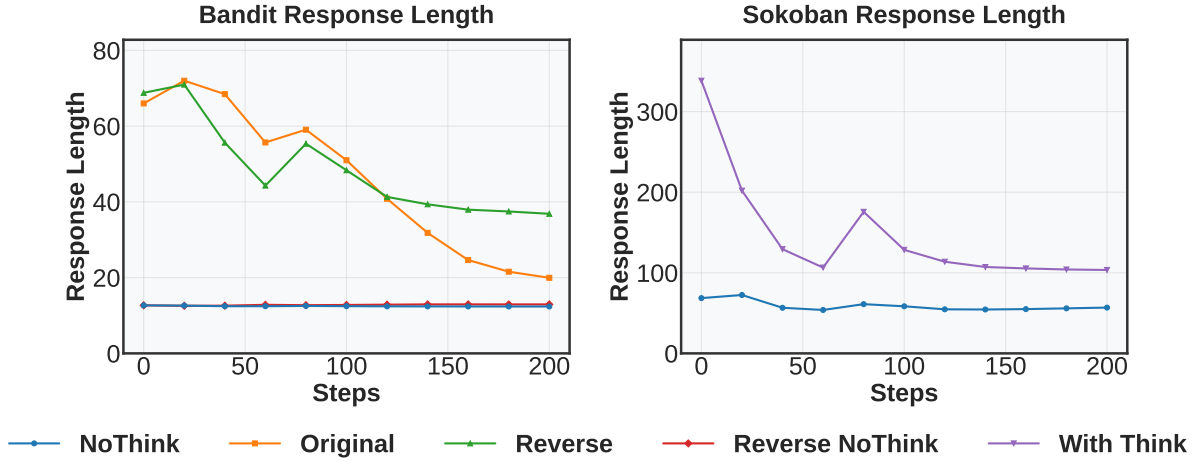


Figure 14 | **Reasoning length over training iterations across different tasks.** We track the average token count of reasoning segments (<think> blocks) during RL training. Across all environments, reasoning length declines as training progresses, with BanditRev maintaining longer traces—possibly due to greater semantic-reward conflict requiring more deliberation.

Lake, agent actions always lead to highly non-deterministic transitions, and intermediate states can appear similar while leading to very different outcomes. This makes value estimation challenging. As PPO relies on a learned value function, instability in critic learning may amplify optimization noise and contribute to early collapse. GRPO, by contrast, does not rely on explicit value learning. Its reward-weighted update procedure may be more tolerant to uncertainty in these settings, leading to comparatively more stable training on Frozen Lake—even if it remains less effective in other tasks. Overall, we summarize environments with high stochasticity may pose greater challenges for value-based methods, and that critic-free approaches can serve as a useful baseline in such cases.

J. Prompt Templates

J.1. Bi-Arm Bandit Environment Prompts

The Bi-Arm bandit environment implements a classic reinforcement learning problem where an agent must balance exploration and exploitation. We present the prompt templates below.

Model Templates

```
<|im_start|>[system]:  
{prompt}  
You're a helpful assistant. You always respond by giving your  
answer in <answer>...</answer>. Max response length: 200  
words (tokens).  
<|im_end|>  
<|im_start|>[user]:  
{prompt}  
You are playing a bandit game. Goal: Maximize your total  
reward by choosing which arm to pull.  
Game Rules:  
1. There are 2 arms, named name_a and name_b  
2. Each arm has its own reward distribution, related to their  
names.  
3. Analyze the symbolic meaning of each arm's name to guess  
how their reward distribution might behave.  
4. Based on the symbolic meaning of their names, which arm do  
you think is more likely to give higher rewards on average?  
Choose between name_a and name_b, and output like <answer>  
name_a </answer> or <answer> name_b </answer>.  
<|im_end|>  
<|im_start|>assistant  
<think>
```

J.2. Sokoban Environment Prompts

The Sokoban environment presents a classic puzzle game where an agent must push boxes to target locations. The following sections detail the prompt structure used to interface with language models.

Model Templates

```
<|im_start|>system
{prompt}
You're a helpful assistant. You always respond by first
wrapping your thoughts in <think>...</think>, then giving your
answer in <answer>...</answer>. Max response length: 200
words (tokens).
<|im_end|>
<|im_start|>user
{prompt}
You are solving the Sokoban puzzle. You are the player and you
need to push all boxes to targets. When you are right next to
a box, you can push it by moving in the same direction. You
cannot push a box through a wall, and you cannot pull a box.
The answer should be a sequence of actions, like <answer>Right
|| Right || Up</answer>
<|im_end|>
<|im_start|>assistant
<think>
```

The environment uses a grid-based representation with specific symbols for different elements:

Grid Representation

The meaning of each symbol in the state is:
#: wall, _: empty, 0: target, ✓: box on target, X: box, P:
player, S: player on target

The instruction template only consists of available actions and restrictions:

Instruction Template

Your available actions are:
Up, Down, Left, Right
You can make up to 10 actions, separated by the action
separator " || "

J.3. FrozenLake Environment Prompts

The FrozenLake environment implements a grid-world navigation task where an agent must traverse a slippery frozen surface to reach a goal. Below we detail the prompt structure used for this environment.

Model Templates

```
<|im_start|>system
{prompt}
You're a helpful assistant. You always respond by first
wrapping your thoughts in <think>...</think>, then giving your
answer in <answer>...</answer>. Max response length: 200
words (tokens).
<|im_end|>
<|im_start|>user
{prompt}
You are solving the FrozenLake puzzle. Forbid the whole and
go to the target. You may move to the unintended direction
due to the slippery ice. Example answer format: <think>To
forbid the hole and go to the target, I should go left then go
up.</think><answer>Left || Up</answer>
<|im_end|>
<|im_start|>assistant
<think>
```

The environment uses a grid-based representation with specific symbols for different elements:

Grid Representation

The meaning of each symbol in the state is:
P: player, _: empty, 0: hole, G: goal, X: player in hole, ✓: player on goal

The instruction template only consists of available actions and restrictions:

Instruction Template

Your available actions are:
Left, Down, Right, Up
You can make up to 10 actions, separated by the action
separator " || "

K. Generalization Evaluation Environments

To evaluate generalization beyond the training distribution, we design two new test environments besides the three training environments that vary along different axes:

- **SokobanDifferentGridVocab** modifies the visual vocabulary used to represent the grid. Instead of using the standard symbols (#, _, O, X, etc.), it maps grid cells to a new vocabulary such as W, G, C, etc. This tests whether the model generalizes across symbol variations while retaining underlying spatial semantics.
- **LargerSokoban** increases the grid size from 6×6 to 8×8 and the number of boxes from 1 to 2, introducing greater spatial complexity and longer-horizon planning demands. This

setting evaluates whether the policy trained on small puzzles can scale up to more complex configurations.

These environments are not seen during training and serve to probe the agent’s generalization capability under symbol shift, size scaling, and environment shift, respectively.

L. Case Study: Spurious Reward from Incorrect Reasoning

While evaluating reasoning behavior in Sokoban, we observe that the model may occasionally receive non-negative or even high rewards despite exhibiting flawed or misleading reasoning traces. Figure 15 presents a 3-turn rollout where the model successfully pushes the box onto the target, yet its intermediate decisions reflect incorrect assumptions about the game dynamics.

In Turn 1 and 2, the agent provides plausible but ultimately incoherent plans, such as "pushing the target" or "moving toward the box on the left," which are either redundant or directionally incorrect. Despite these errors, the final action sequence still manages to reach the goal. Such cases increase reward signal noise, making it harder for RL training to distinguish between genuinely useful plans and coincidentally effective ones.

This highlights a key challenge in multi-turn RL with reasoning agents: *outcome-based reward alone may not sufficiently penalize poor reasoning traces*, especially in environments with sparse or delayed feedback.

M. Extended Case Studies

To better understand how reasoning quality evolves across model scales and environments, we present representative rollout cases across six settings: Bandit (Figure 16), Sokoban (Figure 17), FrozenLake (Figure 18), and WebShop (Figure 19), each at 0.5B and 7B model scales. We observe that **larger models tend to generate longer and more coherent reasoning chains, especially in semantic-rich decision tasks like Bandit and WebShop**. However, in grid-based environments like Sokoban and stochastic ones like FrozenLake, **both small and large models struggle with planning and alignment**, often resorting to brittle heuristics or spurious correlations. These cases align with the experiments in Section D that Sokoban and Frozen Lake does not have a significant performance gain from larger model scales unlike Bandit and WebShop, illustrating how reasoning quality interacts with environment structure, and highlight the challenges of stabilizing reward-grounded reasoning in stochastic or under-specified settings.

Sokoban – Spurious Reasoning Case (3 Turns)

Turn 1:

State:

```
#####  
#__O#  
#_X_#  
###P##  
#####  
#####
```

Output: <think>I’ve reached the target on the right. I need to move right and push the target.</think> <answer>Right || Right || Up</answer> Reward: -0.3

Turn 2:

State:

```
#####  
#_XO#  
#_P_#  
###_##  
#####  
#####
```

Output: <think>Now I need to move left and push the box on the left.</think> <answer>Left || Left || Up</answer> Reward: -0.3

Turn 3:

State:

```
#####  
#P_XO#  
#___#  
###_##  
#####  
#####
```

Output: <think>Now I need to move right and push the box on the right.</think> <answer>Right || Right || Up</answer> Reward: +1.0

Figure 15 | **A rollout with spurious reasoning.** The final outcome is successful, but the reasoning traces across turns are inconsistent and sometimes factually incorrect. This reflects a common failure mode where the model optimizes for final rewards while bypassing coherent reasoning, resulting in noisy and potentially misleading supervision during RL training.

0.5B Bandit
Prompt: Choose the better arm based on symbolic meaning: Dragon vs. Phoenix. Model Output: <think>Dragon has a higher symbolic meaning compared to Phoenix, suggesting it might have a more significant impact on one's character or fate.</think> <answer>Dragon</answer> Reward: +1.0
7B Bandit
Prompt: Choose the better arm based on symbolic meaning: Dragon vs. Phoenix. Model Output: <think>Based on the symbolic meanings of the names "Dragon" and "Phoenix," we can make educated guesses about their reward distributions. The name "Dragon" often symbolizes something powerful, majestic, and long-lived, which could imply a higher average reward and a more stable reward distribution. On the other hand, the name "Phoenix" is often associated with rebirth, transformation, and resilience after destruction or adversity. This could suggest a higher variability in rewards, with the potential for very high rewards but also the risk of lower rewards. Given these interpretations, the Dragon might be more likely to give higher rewards on average, as its name suggests a more powerful and stable source of rewards. However, it's important to note that this is based on symbolic meanings and not on empirical data. In the bandit game, we will need to gather evidence through pulling the arms to make a more informed decision.</think> <answer>Dragon</answer> Reward: +1.0

Figure 16 | **Reasoning-based arm selection in Bandit tasks across model scales.** In both cases, the model must infer reward tendencies of symbolic arms (Dragon vs. Phoenix) based on prior knowledge. The 0.5B model offers a short justification rooted in symbolic association. The 7B model generates a more elaborate reasoning chain comparing stability and variance, reflecting its stronger prior knowledge and interpretive capacity. Both ultimately select Dragon, but through different levels of reasoning depth.

0.5B Sokoban – Multi-Turn Rollout

Turn 1:

State:

```
#####  
#_###  
#0####  
#_XP#  
#_---#  
#####
```

Output:

<think>Pushing all boxes to targets.</think>

<answer>Left || Left || Down</answer>

Reward: -0.3

Turn 2:

State:

```
#####  
#_###  
#0####  
#X_--#  
#_P_--#  
#####
```

Output:

<think>Clearing a wall to make progress.</think>

<answer>Right || Down || Left</answer>

Reward: -0.3

Turn 3:

State:

```
#####  
#_###  
#0####  
#X_--#  
#_P_--#  
#####
```

Output:

<think>Pushing another box.</think>

<answer>Left || Down || Up</answer>

Reward: -0.3

7B Sokoban – Multi-Turn Rollout

Turn 1:

State:

```
#####  
##_P_#  
##_X_#  
##_O#  
###_#  
#####
```

Output:

<think>First, move the box to target.</think>

<answer>Up || Right || Up</answer>

Reward: -0.3

Turn 2:

State:

```
#####  
##_P#  
##_X_#  
##_O#  
###_#  
#####
```

Output:

<think>Now move second box to target.</think>

<answer>Left || Down || Left || Down</answer>

Reward: -0.4

Turn 3:

State:

```
#####  
##_#  
##_#  
##PXO#  
###_#  
#####
```

Output:

<think>Finally, move last box to target.</think>

<answer>Right || Up || Right || Up</answer>

Reward: +1.0

Figure 17 | **Sokoban rollouts across model scales.** At 0.5B, the model performs minimal reasoning and often issues locally valid but suboptimal actions. The 7B model demonstrates more structured planning and symbolic alignment across turns, though still exhibits inefficiencies and heuristic moves in long-horizon settings.

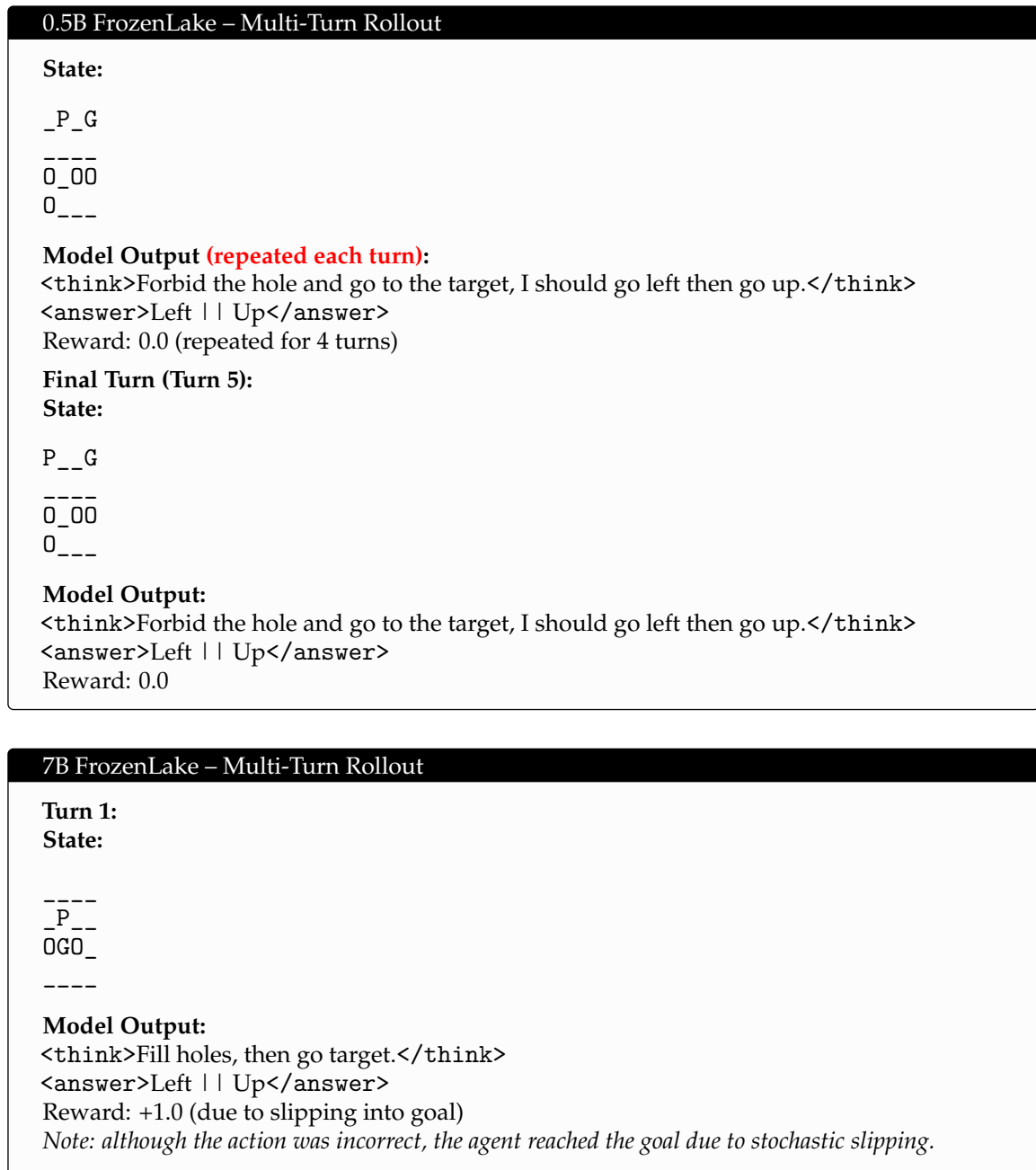


Figure 18 | **FrozenLake rollouts across model scales.** At 0.5B, the agent repeats a fixed plan regardless of outcome, suggesting limited adaptation or planning. The 7B agent receives high reward despite issuing a suboptimal command, due to stochastic transitions—highlighting the difficulty of credit assignment in such environments and the risk of reinforcing spurious patterns.

0.5B WebShop – Multi-Turn Rollout

Turn 1:

State: Instruction: Find me super soft decorative pillows for living room with color: beige latte, and size: 20"x20", and price lower than 30.00 dollars. We must buy a product within 10 actions. It doesn't have to match perfectly with description. Search term should not include details like size, ...

Model Output:

<think>To find a super soft decorative pillows for the living room with a beige latte color and a size of 20"x20", I will start with a search for pillows with a beige color and a size of 20"x20".</think>

<answer>search[bisque pillow with beige color 20"x20']</answer>

Reward: 0.0

Turn 2:

State: Instruction: [SEP] Find me super soft decorative pillows for living room with color: beige latte, and size: 20"x20", and price lower than 30.00 dollars [SEP] Back to Search [SEP] Page 1 (Total results: 50) [SEP] Next > [SEP] B08L2ZDWN2 [SEP] PAVILIA Decorative Sherpa Throw Pillow Covers, ... click[b08qvdnjc7], click[b07p24zmxk], click[b07n33bpq6], click[b09nrf2qgd],

Model Output:

<think>Searching for pillows with beige color and 20"x20" size, and price lower than \$30.</think>

<answer>click[b08l2zdown2]</answer>

Reward: 0.0

Turn 3:

State: Instruction: [SEP] Find me super soft decorative pillows for living room with color: beige latte, and size: 20"x20", and price lower than 30.00 dollars [SEP] Back to Search [SEP] < Prev [SEP] color [SEP] beige latte [SEP] black [SEP] blush pink [SEP] dusty blue [SEP] emerald green [SEP] ... grey], click[mustard yellow], click[teal blue], click[white], click[18"x18"],

Model Output:

<think>Searching for pillows with beige color and 20"x20" size, and price lower than \$30.</think>

<answer>click[beige latte]</answer>

Reward: 0.0 ...

Turn 9:

State: Instruction: [SEP] Find me super soft decorative pillows for living room with color: beige latte, and size: 20"x20", and price lower than 30.00 dollars [SEP] Back to Search [SEP] < Prev [SEP] color [SEP] beige latte [SEP] black [SEP] blush pink [SEP] dusty blue [SEP] emerald green [SEP] ... grey], click[mustard yellow], click[teal blue], click[white], click[18"x18"],

Model Output:

<think>Searching for pillows with beige color and 20"x20" size, and price lower than \$30.</think>

<answer>click[black]</answer>

Reward: 0.0

3B WebShop – Multi-Turn Rollout

Turn 1:

State: Instruction: Find me wall mounted, white item, easy clean tv stands for living room with color: white+brown, and price lower than 130.00 dollars. We must buy a product within 10 actions. It doesn't have to match perfectly with description. Search term should not include details like size, ...

Model Output:

<think>Let's start by searching for the items within the given criteria and narrow down the options. The search term will be 'white+brown tv stand living room' to find wall-mounted, easy-clean TV stands with the specified colors and in the living room category.</think>

<answer>search[white+brown tv stand living room]</answer>

Reward: 0.0

Turn 2:

State: Instruction: [SEP] Find me wall mounted, white item, easy clean tv stands for living room with color: white+brown, and price lower than 130.00 dollars [SEP] Back to Search [SEP] Page 1 (Total results: 50) [SEP] Next > [SEP] B09F6SCW47 [SEP] hongxing Floating TV Stand with Storage Shelves, ... click[b079n3v1rj], click[b0924pklq4], click[b09lhm4wqs], click[b08k7ldm7q],

Model Output:

<think>After reviewing the search results, the first item that seems to match the criteria is the 'Floating TV Stand with Storage Shelves, Chipboard Wall Mounted Hanging Television Cabinet' from 'B09F6SCW47'. It is white with brown accents, fits the living room and bedroom category, and is priced at \$90.79, which is lower than the \$130.00 limit. It's also the closest match in terms of color and style. Since it's the first matching option, I will prioritize clicking on it to move forward with the purchase. However, I'll make sure to click [size] and then [color] if they are available and required to finalize the purchase.</think>

<answer>click[b09f6scw47]</answer>

Reward: 0.0

...

Turn 7:

State: Instruction: [SEP] Find me wall mounted, white item, easy clean tv stands for living room with color: white+brown, and price lower than 130.00 dollars [SEP] Back to Search [SEP] < Prev [SEP] color [SEP] brown [SEP] concrete gray [SEP] grey [SEP] white [SEP] white+brown [SEP] hongxing ... click[buy now], click[brown], click[concrete gray], click[grey], click[white],

Model Output:

<think>With the 'Floating TV Stand with Storage Shelves, Chipboard Wall Mounted Hanging Television Cabinet' from 'B09F6SCW47' confirmed to have the correct white and brown color options and the required size of 12Inx11.8Inx35.4In, I will now proceed to click the 'buy now' button to finalize the purchase. Given that we have 3 actions left and the product matches the criteria closely, I will click on 'buy now' to complete the transaction. This ensures that we stay within the action limit and make a purchase that closely aligns with the specified requirements.</think>

<answer>click[buy now]</answer>

Reward: 1.0

Figure 19 | WebShop rollouts illustrate the impact of model scale on long-context decision making. At 0.5B, the agent becomes stuck in a loop, repeatedly selecting irrelevant options despite rich contextual information— indicating difficulty in long-horizon memory and goal tracking. In contrast, the 3B model executes a successful multi-step reasoning chain: narrowing search queries, navigating product options, selecting attributes, and finalizing purchase. This highlights the importance of scale for compositional planning in realistic, open-domain environments.